Designing a RISC-V CPU in VHDL

Laboratory manual – RV32I from ALU to graphics on an FPGA

Student edition

Specification version: Unprivileged ISA 20250508
Manual revision: October 2026

Contents

About this manual **2**

0.1 How the labs fit together 2

0.2 Structure of every lab 2

0.3 Reference documents 3

0.4 Diagrams 3

0.5 Repository and tools 3

0 Environment and workflow **5**

1 The RV32I instruction set and the ALU **10**

2 Register file and immediate generator **15**

3 Decode and control **20**

4 Integration: a working core **25**

5 The software toolchain **32**

6 Verification in depth **37**

7 FPGA deployment **42**

8 Graphics on the CPU **48**

A The portable VHDL subset **54**

B Memory map and I/O registers **56**

C Reference documents and versions **57**

About this manual

Over nine laboratory sessions you will design, verify and deploy a complete 32-bit RISC-V processor (the RV32I base instruction set) in VHDL, run C programs on it, and finish by drawing graphics on a VGA monitor from an FPGA board. Every block is yours: this manual gives you interfaces, constraints, timing diagrams and hints, and the RISC-V specification gives you the rules. Nothing in this manual gives you a complete architecture or a control table to copy.

0.1 How the labs fit together

Lab	You build	You prove it with
0	tool installation, a first cocotb test, a bitstream	the provided counter test
1	the ALU, and your first Python reference model	directed and random tests
2	the register file and the immediate generator	directed, encoded and random tests
3	the main decoder and the ALU control	architectural-effect tests
4	the complete core: PC logic, branches, loads and stores, memories	instruction-by-instruction traces
5	the software side: C start-up code, linking, memory images	C programs on your core
6	a golden model and a verification environment	lockstep, random programs, coverage, riscv-arch-test
7	the system on a board: clocking, reset, I/O	board wrappers in simulation, then hardware
8	a VGA framebuffer and graphics software	pixel-exact golden images, cycle counts

The software toolchain (Lab 5) comes *before* verification in depth (Lab 6) because the RISC-V architecture tests and lockstep runs on real programs need compiled ELF files, a linker script and the simulation-only I/O registers that Lab 5 introduces.

0.2 Structure of every lab

Each chapter follows the same pattern:

Objectives
: what you will be able to do, in measurable terms.

Pre-lab reading
: specific sections of the specifications, cited by section *title* (section numbers change between releases). Read them *before* the session.

Theory
: at most about two pages: block diagrams and a few *design rules*.

Spec lookup questions
: questions you can answer only by reading the specification. Write your answers in your lab book; they feed straight into your design.

Tasks
numbered and in order. Each has a "done when" condition: do not move on until it holds.

Verification checkpoint
the tests that must pass. Some are provided; some you write.

Deliverables and marking
what to submit and how it is assessed.

Extension
optional work for those who finish early.

Four kinds of box appear throughout:

Design rule

A rule your design must follow. Breaking a design rule is a design error even if a test happens to pass.

Spec lookup

Questions answered by the specification, not by this manual.

Verification checkpoint

The tests that must pass before you move on.

Warning

A common trap, with the reason it catches people.

0.3 Reference documents

All specification references in this manual are to these exact versions (details in Appendix C):

- *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508 – "the spec".
- *RISC-V ABIs Specification* (ELF psABI), Version 1.0 – calling convention, register names.
- *RISC-V Assembly Programmer's Manual* (riscv-non-isa/riscv-asm-manual, commit `167ad3d`) – assembler syntax, pseudo-instructions.

0.4 Diagrams

Every diagram uses the conventions in Figure 1. Datapath signals are blue and solid, control signals are orange and dashed, and a slash with a number gives a bus width in bits.

0.5 Repository and tools

You receive a starter repository. Its `README.md` explains how to install the tools on Rocky Linux 9 or Windows 10/11 and how to run each lab's tests. In short:

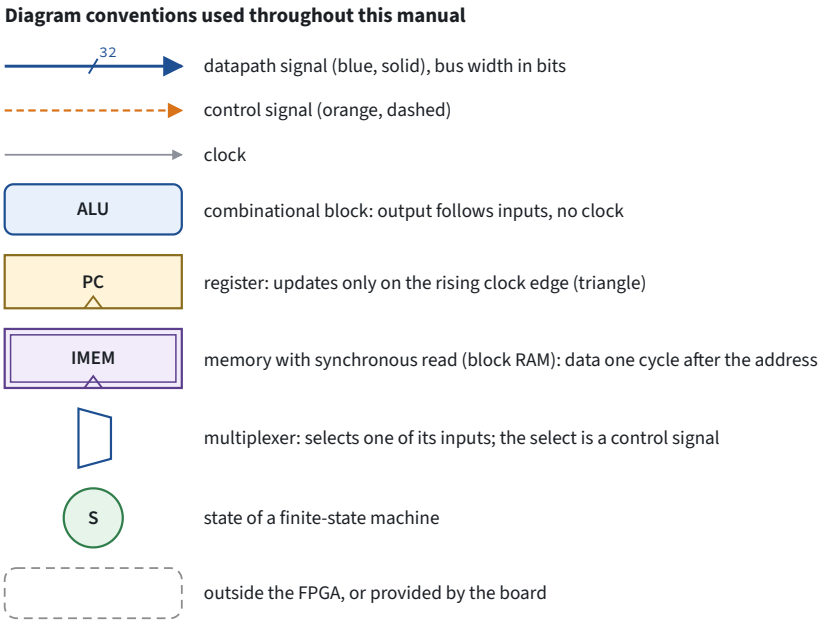


Figure 1: Diagram conventions used throughout this manual.

- `rtl/`, `boards/` – your VHDL. Entities and port lists are given and must not change; architectures are yours.
- `tests/labNN/` – cocotb tests. Run them with `make test LAB=N` (GHDL) or `make test LAB=N SIM=nvc`.
- `sw/` – C and assembly programs, built with `make`.
- `sim/` – simulation-only VHDL (testbenches, models). Never add these files to a synthesis project.

LAB 0

Environment and workflow

Objectives

By the end of this lab you will be able to:

1. install and check the simulation, software and FPGA tools on your computer;
2. run a cocotb test against a VHDL design with GHDL and NVC, and read its pass/fail report and waveform;
3. create a vendor project, synthesise a design, read its resource and timing summary, and program your board;
4. explain the difference between what a simulation proves and what synthesis and timing analysis prove.

Pre-lab reading

- The repository's `README.md`: installation for your operating system and board.
- *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508: chapter *"Introduction"*, sections *"RISC-V ISA Overview"* and *"RISC-V Hardware Platform Terminology"*. You will not need the details yet, but you should know what a *hart*, a *base ISA* and an *extension* are.
- The cocotb documentation's quick-start page (linked from the README).

Theory

Hardware design uses two separate flows (Figure 2). In *simulation*, a simulator (GHDL or NVC) runs your VHDL as a program and a testbench checks its behaviour. In *implementation*, a vendor tool (Vivado for the Basys 3, Quartus Prime Lite for the DE10-Lite) turns the same VHDL into a circuit, places it on the FPGA, and checks that every path meets the clock period.

The two flows check different things. A design can pass every simulation and still fail on the board, for example if it does not meet timing, if a pin is wrong, or if it relies on a simulation-only construct. A design that synthesises cleanly can still be functionally wrong. You need both.

In this course the testbenches are written in Python with **cocotb**: the simulator runs the VHDL, and cocotb lets Python drive the design's inputs, wait for clock edges and read its outputs (Figure 3). Python gives you a full programming language for checking: reference models, random stimulus and coverage. You will use all of these by Lab 6.

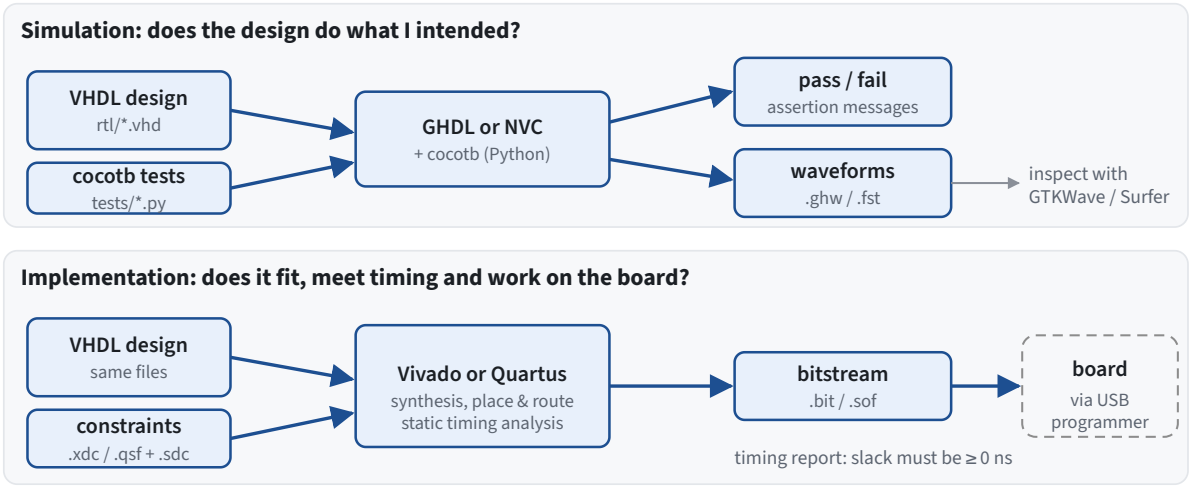


Figure 2: Simulation and implementation use the same VHDL but answer different questions.

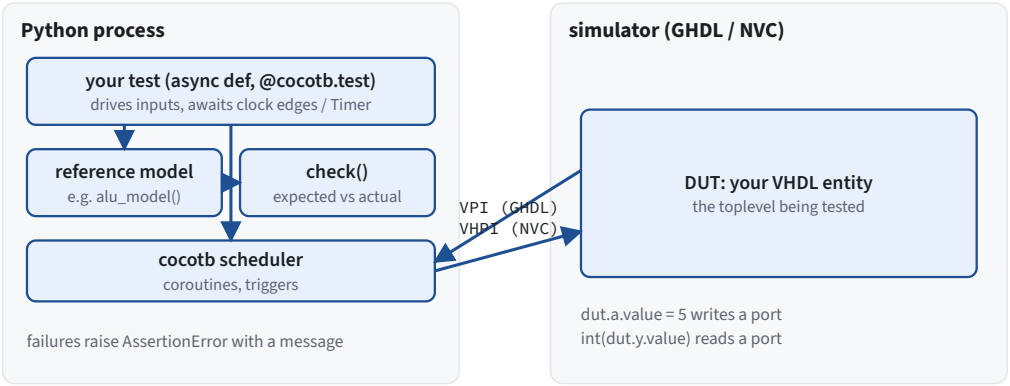


Figure 3: A cocotb test runs in Python and talks to the simulator through its VPI (GHDL) or VHPI (NVC) interface.

Design rule

Vivado's built-in simulator (xsim) is *not* used: cocotb does not support it. Vendor tools are for synthesis, implementation and programming only. Intel users may optionally use Questa for debugging, but the assessed tests run on GHDL or NVC.

The provided design is a counter (Figure 4). It is small enough to read completely: one register, an incrementer and an enable.

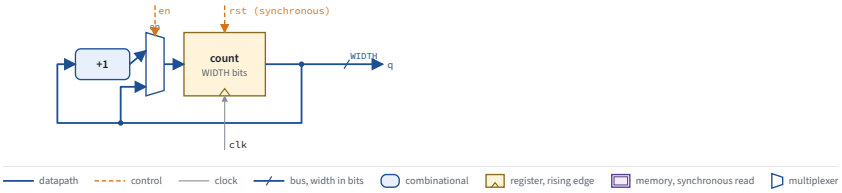


Figure 4: The Lab 0 counter: `lab0/counter.vhd`.

Spec lookup questions

Spec lookup

1. In the spec's terminology, what is the difference between a *core* and a *hart*? How many harts will your CPU have?
2. The spec says RISC-V is a *family* of ISAs. Name the base integer ISA you will implement, and explain in one sentence what "base" means.
3. What is the difference between a *ratified* and a *frozen* or *draft* part of the specification? Which version of the unprivileged specification does this manual use, and is it ratified?

Tasks

Task 0.1 Install the tools

Follow `README.md` for your operating system. You need Python 3.10 or later with the packages in `requirements.txt`, GHDL and NVC, GNU Make, the xPack `riscv-none-elf-gcc` toolchain, and the vendor tool for your board.

Done when: `ghdl --version`, `nvc --version`, `riscv-none-elf-gcc --version` and `python3 -c "import cocotb; print(cocotb.__version__)"` all print a version, and you have recorded the versions in your lab book.

Task 0.2 Run the provided test

In the repository root run `make test LAB=0`, then `make test LAB=0 SIM=nvc`. Read `tests/lab00/test_counter.py`: find where the clock is started, where the reset is applied and where each

value is checked.

Done when: both runs end with `1 passed`: pytest counts one item per simulation run, and that run contains the three cocotb tests `counts_when_enabled`, `holds_when_disabled` and `wraps_around`. To see cocotb's per-test table, run `make test LAB=0 PYTEST_ARGS=-s`; it is also saved as `sim_build/<sim>/<toplevel>/*.result.xml`.

Task 0.3 Break it on purpose

Change `lab0/counter.vhd` so that the counter counts by two, and run the test again. Read the failure message. Then run `make test LAB=0 WAVES=1`, open the waveform (`.ghw` with GTKWave or Surfer) from `sim_build/ghdl/counter/`, and find the cycle where the test fails. Restore the file.

Done when: you can point at the failing cycle in the waveform and explain how the failure message identifies it.

Task 0.4 Synthesise and program

Create the project with the script for your board:

```
quartus_sh -t scripts/quartus/create_lab0.tcl compile                  # DE10-Lite
vivado -mode batch -source scripts/vivado/create_lab0.tcl -tclargs build   # Basys 3
```

Open the project in the vendor GUI. Find the resource summary (how many registers and LUTs or logic elements?) and the timing summary (what is the worst slack?). Program the board. Switch 0 enables the counter and the reset button (KEY0 or btnC) clears it.

Done when: the LEDs count on your board, and you have recorded the resource use and the worst slack.

Verification checkpoint

Verification checkpoint

`make test LAB=0` passes under GHDL and NVC, and the counter runs on your board.

Deliverables and marking

- Tool versions, and the answers to the spec lookup questions (2 marks).
- Screenshot or log of both test runs passing (2 marks).
- The failing waveform from Task 0.3, annotated (2 marks).
- Resource use and worst slack from the vendor report, with one sentence explaining what a positive slack means (2 marks).
- Demonstration on the board (2 marks).

Extension

Make the LED pattern change speed with switches 1 to 3 (for example, choose which slice of the counter drives the LEDs). Extend the cocotb test so that it checks the new behaviour, by testing the VHDL with a smaller `WIDTH` generic.

LAB 1

The RV32I instruction set and the ALU

Objectives

By the end of this lab you will be able to:

1. identify the six RV32I instruction formats and the fields each contains;
2. sketch the single-cycle datapath at block level and say which block each instruction class uses;
3. implement a combinational 32-bit ALU that synthesises in both Vivado and Quartus;
4. write a Python reference model from the specification and use it in a constrained-random cocotb test of at least 2000 operations.

Pre-lab reading

From *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508, chapter *"RV32I Base Integer Instruction Set, Version 2.1"*:

- *"Programmers' Model for Base Integer ISA"*
- *"Base Instruction Formats"*
- *"Integer Computational Instructions"*, with both subsections *"Integer Register-Immediate Instructions"* and *"Integer Register-Register Operations"*

Also skim the RV32I table in chapter *"RV32/64G Instruction Set Listings"*: you will use it constantly from now on.

Theory

Instruction formats

Every RV32I instruction is 32 bits long. The spec defines six formats (Figure 5). Their key property is that the register specifiers `rs1`, `rs2` and `rd` are always in the same bit positions, so the register file can start reading before the instruction has been decoded. The immediate bits move around between formats; in the B and J formats they are scrambled in a way you will look up in Lab 2.

The single-cycle datapath

Figure 6 shows the CPU you will build, at block level. In each cycle the PC addresses the instruction memory; the instruction's fields read two registers and form an immediate; the ALU

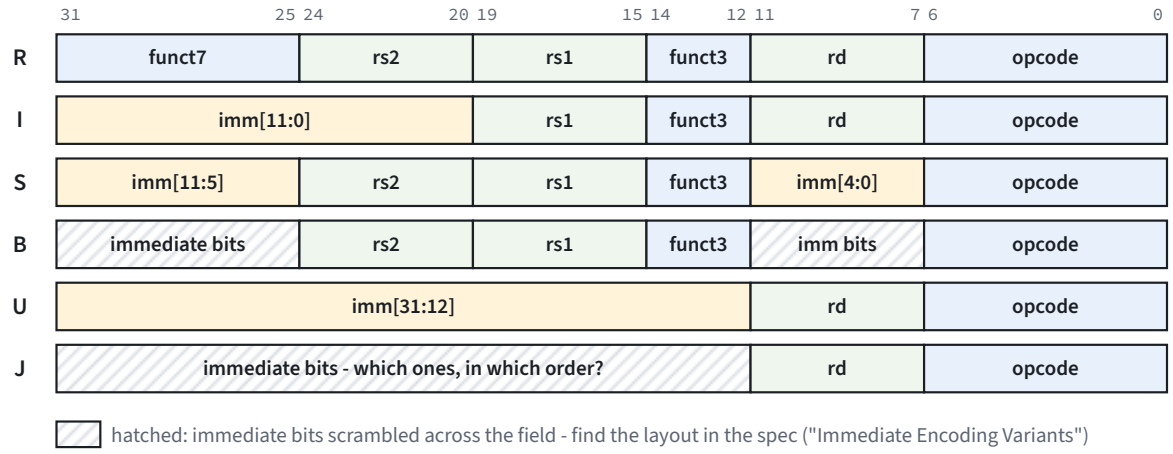


Figure 5: RV32I instruction formats. Hatched fields hold immediate bits whose order you must find in the spec.

combines two operands; the result is written back, used as a data-memory address, or used as a branch target. Labs 1 to 3 build the blocks; Lab 4 joins them and deals with the fact that FPGA memories are synchronous.

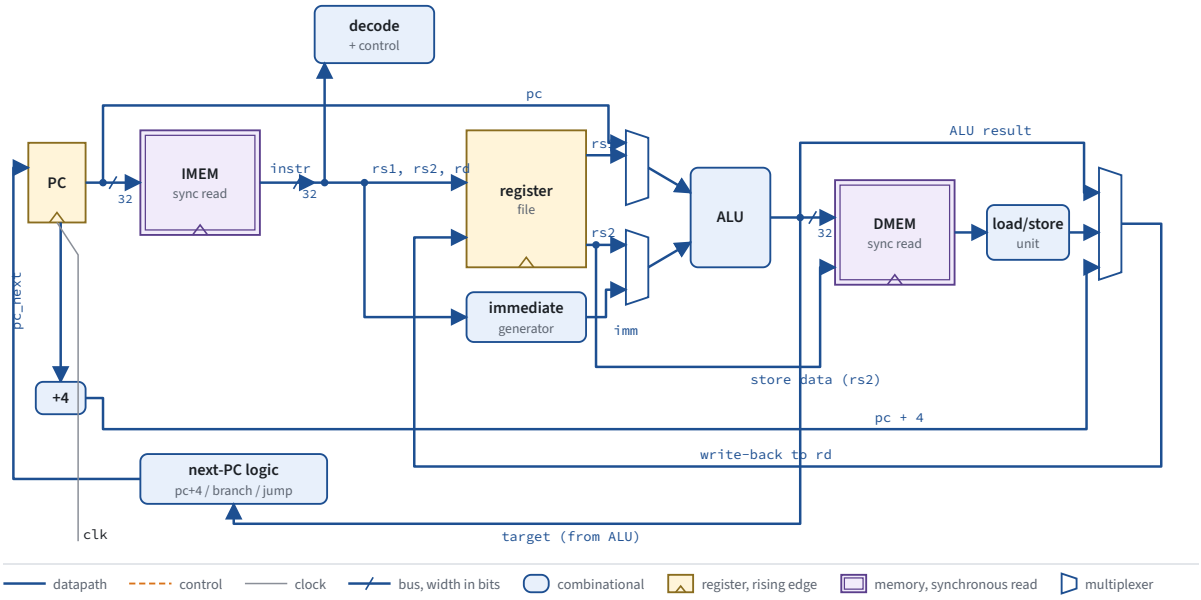


Figure 6: Block-level datapath. Control signals are omitted here; they are the subject of Lab 3.

The ALU

The ALU (Figure 7) computes one of ten operations on two 32-bit operands. It is purely combinational. The operation encodings are fixed in `rtl/core/cpu_pkg.vhd` (`ALU_ADD`, `ALU_SUB`, ...). They are deliberately *not* the same as any instruction field, so in Lab 3 you will have to work out how each instruction selects its operation.

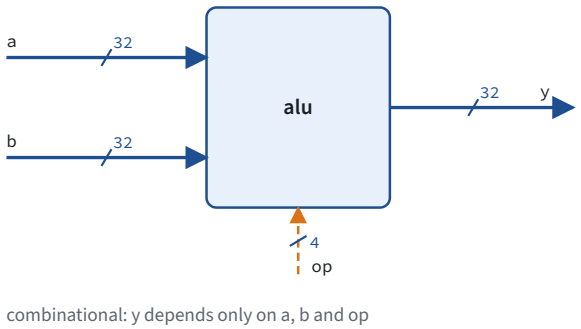


Figure 7: ALU interface (`rtl/core/alu.vhd`).

Design rule

One driver per signal, every output on every path. A combinational process must assign every output on every path through it. A missing assignment infers a latch, which is a design error. Give outputs a default at the top of the process, or cover every case.

Warning

Every test compiles every RTL file. The test runner analyses all of `rtl/sources.f`, not just the block under test. Keep every file analysable, including the ones you have not started: an unfinished `decoder.vhd` with a syntax error breaks `make test LAB=1`. Check with `make elab`.

Design rule

Use `numeric_std` only. Convert with `unsigned()` and `signed()`; never use `std_logic_arith` or `std_logic_unsigned`. Arithmetic on `std_logic_vector` without a conversion says nothing about whether the value is signed.

Design rule

Stay inside the portable VHDL-2008 subset in Appendix A. `process(all)` is allowed; check anything else against the appendix before you rely on it. Quartus Prime Lite's VHDL-2008 support is partial.

Testing against a model

A test that checks a few hand-picked values is a good start, but it only covers what you thought of. The stronger technique is a *reference model*: an independent Python function that computes the expected result for *any* input. Random inputs are then checked automatically. The model must be written from the specification, not from your VHDL; otherwise it repeats your mistakes. Random does not mean uniform: values such as 0, -1, `0x80000000` and `0x7FFFFFFF` find most arithmetic bugs, so bias your stimulus towards them.

Spec lookup questions

Spec lookup

1. Which register is hard-wired to zero, and what happens to a value written to it?
2. In which bit positions are `rs1`, `rs2` and `rd` in every format that has them? Why does the spec say this matters?
3. What is the difference between `SRL` and `SRA`? Which instruction bit distinguishes them?
4. For register-register shifts, which bits of `rs2` give the shift amount? What happens to the others?
5. What does `SLTIU rd, rs1, 1` compute? (The spec names the pseudo-instruction.) How is the immediate extended for `SLTIU`, and which unsigned constants can it therefore compare against?
6. Search the RV32I table in *"RV32/64G Instruction Set Listings"*: is there a subtract-immediate instruction? Which instruction subtracts a constant instead?
7. How does the spec define arithmetic overflow for `ADD`, and what does it say software should do about it?

Tasks

Task 1.1 Read the formats

Using the spec, write the six formats in your lab book. For each RV32I instruction class (register-register, register-immediate, load, store, branch, upper immediate, jump) give its format and say which blocks of Figure 6 it uses.

Done when: your table covers every instruction in the RV32I listing.

Task 1.2 Plan the ALU

For each ALU operation in `cpu_pkg.vhd`, write how you will compute it with `numeric_std`, and which RV32I instructions will use it. Decide what the ALU outputs for an undefined `op` value: it must be a defined 0/1 value, never `'X'` or `'-'`.

Done when: every one of the ten operations has a planned implementation.

Task 1.3 Implement the ALU

Fill in the architecture in `rtl/core/alu.vhd`. Run `make test LAB=1 TESTCASE=alu_directed`.

Done when: `alu_directed` passes under GHDL and NVC.

Task 1.4 Write the reference model and a random test

Implement `alu_model()` in `tests/common/labtest/models.py` from the specification. Then implement `alu_random` in `tests/lab01/test_alu.py`: at least 2000 operations with a fixed seed, operands biased towards edge values. Check that your test can fail: break the ALU on purpose

(for example, make `SRA` behave like `SRL`) and make sure `alu_random` catches it.

Done when: `make test LAB=1` passes, and you have recorded the message `alu_random` gave for your deliberate bug.

Task 1.5 Edge cases

Add a test that checks every operation on every pair from a list of at least eight edge values, and a test for shift amounts 0 to 31 with the upper bits of `b` set.

Done when: both new tests pass (name them, for example, `alu_edge_pairs` and `alu_shift_amounts`), and you can explain which bug each one targets.

Task 1.6 Synthesis check

Run your ALU through your vendor tool on its own (make `alu` the top level of a scratch project) or through `ghdl --synth`. Look for latch warnings.

Done when: no latches are reported.

Verification checkpoint

Verification checkpoint

`make test LAB=1` passes under GHDL and NVC: `alu_directed` (provided), `alu_random` and your edge-case tests.

Deliverables and marking

- Answers to the spec lookup questions and the format table from Task 1.1 (3 marks).
- `alu.vhd`: correct, synthesisable, no latches, readable (3 marks).
- `models.py` and `test_alu.py`: an independent model, biased random stimulus, a fixed seed, edge-case tests (3 marks).
- Evidence that your test detected a deliberate bug (1 mark).

Extension

Add a `zero` output and a carry/overflow output, and test them. Discuss in a paragraph why RISC-V does not need a flags register for branches, and what that means for how you will compare values in Lab 4.

LAB 2

Register file and immediate generator

Objectives

By the end of this lab you will be able to:

1. implement a 32 × 32-bit register file with two combinational read ports, one clocked write port and a hard-wired zero register;
2. explain, with resource estimates for both boards, why the register file uses asynchronous reads while the memories use synchronous reads;
3. decode the I, S, B, U and J immediates from the spec's encoding figure, including the scrambled B and J layouts;
4. write an immediate encoder and decoder in Python and use them for encoded-extreme and random tests.

Pre-lab reading

From *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508, chapter *"RV32I Base Integer Instruction Set, Version 2.1"*:

- *"Programmers' Model for Base Integer ISA"*
- *"Base Instruction Formats"* and *"Immediate Encoding Variants"* (read the figure of immediate types bit by bit)
- *"Control Transfer Instructions"*: only the paragraphs that say how branch and jump offsets are formed.

From the *RISC-V ABIs Specification* (ELF psABI), Version 1.0: *"Integer Register Convention"* (the ABI names `ra`, `sp`, `a0` and so on).

Theory

The register file

RV32I has 31 general-purpose registers plus `x0`, which always reads as zero. The register file (Figure 8) has two read ports, because an instruction reads up to two sources, and one write port.

The key design decision is *when* the read data appears (Figure 9). With an *asynchronous* read the data follows the address within the same cycle, which a single-cycle datapath needs: the instruction arrives, its `rs1` field selects a register, and the value must reach the ALU before the cycle ends. FPGA block RAM only offers a *synchronous* read: the address is registered at a clock edge and the data appears in the next cycle.

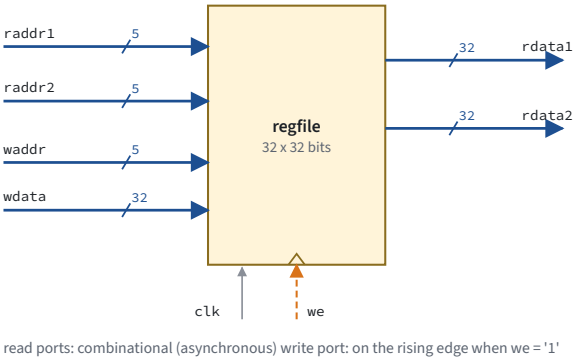


Figure 8: Register file interface (`rtl/core/regfile.vhd`).

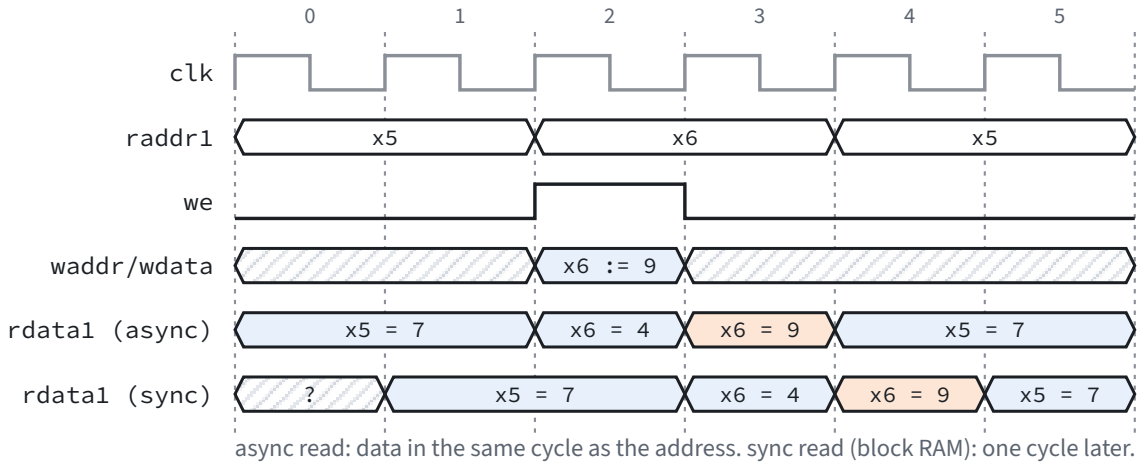


Figure 9: Asynchronous and synchronous read of the register file, including a read of a register that is written in the same cycle.

This choice has different costs on the two boards:

- The Artix-7 has *distributed RAM* (LUT-RAM): look-up tables used as small asynchronous-read memories. A 32 × 32 two-read-port register file costs about a dozen RAM32M blocks.
- The MAX 10 has no LUT-RAM. An asynchronous-read register file becomes 1024 flip-flops plus two 32-way 32-bit multiplexers, which is roughly 2000 logic elements, still small on a 10M50.
- Using block RAM would save logic on both devices, but it would add a cycle of latency to every register read. That would turn the design into a pipeline (see the instructor-led discussion in Lab 4).

Design rule

All state changes on the rising edge. The only clocked process in the register file is the write. Reads are combinational. Do not write on the falling edge to "get the value earlier"; the timing tools then have only half a cycle for every path through the register file.

Design rule

`x0` is hard-wired to zero. Whatever is written to it, reading it gives zero. Decide whether you enforce this on the read side, the write side or both, and justify it. Note that it affects what synthesis can infer.

The immediate generator

An instruction's immediate is a constant stored in its encoding. The immediate generator (Figure 10) extracts those bits, puts them in order and sign-extends them to 32 bits (Figure 11). It does not know which instruction it is looking at; the decoder tells it which format to use, through `imm_sel` (Lab 3).

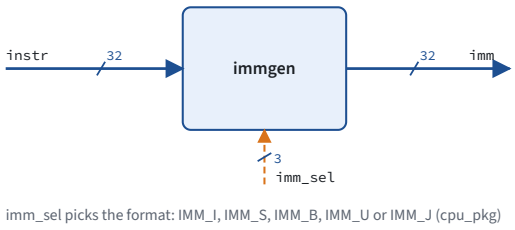


Figure 10: Immediate generator interface (`rtl/core/immgen.vhd`).

Spec lookup questions

Spec lookup

1. In the B format, which instruction bits hold `imm[12]`, `imm[11]`, `imm[10:5]` and `imm[4:1]`? Why is `imm[0]` not stored?
2. In the J format, which instruction bits hold `imm[20]`, `imm[19:12]`, `imm[11]` and `imm[10:1]`?
3. The spec explains why the immediate bits are scrambled. Give its reasons, in your own

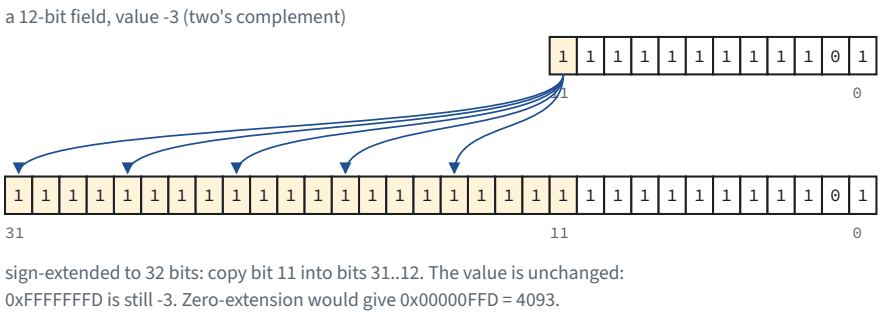


Figure 11: Sign extension keeps the value of a two's-complement number when it is widened.

words, in terms of the hardware you are building today.

4. Which instruction bit is the sign bit of every immediate?
5. What range of byte offsets can a conditional branch reach? A `JAL`?
6. What does a U-type immediate put in bits 11..0 of the result?
7. (Reasoning, from the `ADDI` description.) `LUI` followed by `ADDI` builds any 32-bit constant. Why must the value given to `LUI` sometimes be one more than the top 20 bits of the constant? Give an example.
8. What are the ABI names of `x1`, `x2` and `x8`?

Tasks

Task 2.1 Trade-off analysis

Before writing VHDL, fill in a table with rows {Artix-7, MAX 10} and columns {asynchronous read, synchronous read}. In each cell give the resource you expect synthesis to use, an estimate of how much, and the consequence for the datapath.

Done when: all four cells are filled and you can explain why this lab uses asynchronous reads.

Task 2.2 Implement the register file

Fill in `rtl/core/regfile.vhd`. Run `make test LAB=2 K=regfile` (the six provided tests in `tests/lab02/test_regfile.py`).

Done when: all six register-file tests pass under GHDL and NVC.

Task 2.3 Answer the spec lookup questions

Do not skip this: you need the answers for the next task.

Done when: your answers to every question are in your lab book.

Task 2.4 Implement the immediate generator

Fill in `rtl/core/immgen.vhd` from the spec figure. Run `make test LAB=2 K=immgen TESTCAS`

`E=imm_directed`.

Done when: `imm_directed` passes.

Task 2.5 Encoder and decoder model

Implement `decode_imm` and `encode_imm` in `tests/common/labtest/models.py`. Then implement `imm_encoded_extremes`: for every format, encode the most negative, most positive and a few patterned immediates (for example alternating bits), drive the word into the DUT and check that the generator recovers each one.

Done when: `imm_encoded_extremes` passes, and `encode_imm` raises `ValueError` for an odd B/J immediate and an out-of-range value.

Task 2.6 Random immediate test

Implement `imm_random`: random 32-bit words in every format, checked against `decode_imm`, plus a round-trip check `decode_imm(encode_imm(fmt, v), fmt) == v & MASK32` for random encodable values (`decode_imm` returns an unsigned 32-bit pattern).

Done when: `make test LAB=2` passes in full under both simulators.

Task 2.7 Check what synthesis built

Synthesise `regfile` on its own and find, in the report, what the storage became (LUT-RAM, registers or block RAM).

Done when: your lab book compares the report with your prediction in Task 2.1.

Verification checkpoint

Verification checkpoint

`make test LAB=2` passes under GHDL and NVC: six provided register-file tests, `imm_directed` (provided), and your `imm_encoded_extremes` and `imm_random`.

Deliverables and marking

- Spec lookup answers and the trade-off table (3 marks).
- `regfile.vhd` and `immgen.vhd`, correct and synthesisable (3 marks).
- `models.py` encoder and decoder with the two tests (3 marks).
- The synthesis inference comparison (1 mark).

Extension

Add a third read port to the register file (useful for a store that also reads its base address in a pipeline). Measure the cost in your vendor's report, and explain why the cost differs between the two FPGAs.

LAB 3

Decode and control

Objectives

By the end of this lab you will be able to:

1. derive, from the spec alone, the value of every datapath control signal for every RV32I instruction, including the "don't care" entries;
2. derive the second-level ALU-control table from `funct3` and `funct7`;
3. implement both as combinational VHDL that flags every unimplemented encoding as illegal without requesting any state change;
4. test a control unit by its architectural effect, without writing its table into the test.

Pre-lab reading

From *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508, chapter *"RV32I Base Integer Instruction Set, Version 2.1"*: the whole of *"Integer Computational Instructions"*, *"Control Transfer Instructions"*, *"Load and Store Instructions"*, *"Memory Ordering Instructions"*, *"Environment Call and Breakpoints"* and *"HINT Instructions"*. Also read the RV32I base table in *"RV32/64G Instruction Set Listings"*: every opcode, `funct3` and `funct7` you need is there.

Theory

Two-level decoding

The decoder turns an instruction word into the signals that steer the datapath. Most of the work depends only on the opcode: which operands the ALU sees, whether a register is written, whether memory is accessed. The exact ALU operation depends on `funct3` and `funct7` as well. Splitting the decoder into a main decoder and an ALU-control unit (Figure 12) keeps each table small. The main decoder classifies the instruction (`alu_class`); the ALU control finishes the job.

Control points

Figure 13 shows where each control signal acts. The meaning of each signal and of each encoding is defined in `cpu_pkg.vhd` and in the port comments of `decoder.vhd`. For example, `src_a` selects `rs1`, the PC or zero as ALU operand A, and `res_src` selects what is written back to `rd`. For each instruction you decide which value each signal takes and when it does not matter.

Some things to notice while you derive the table:

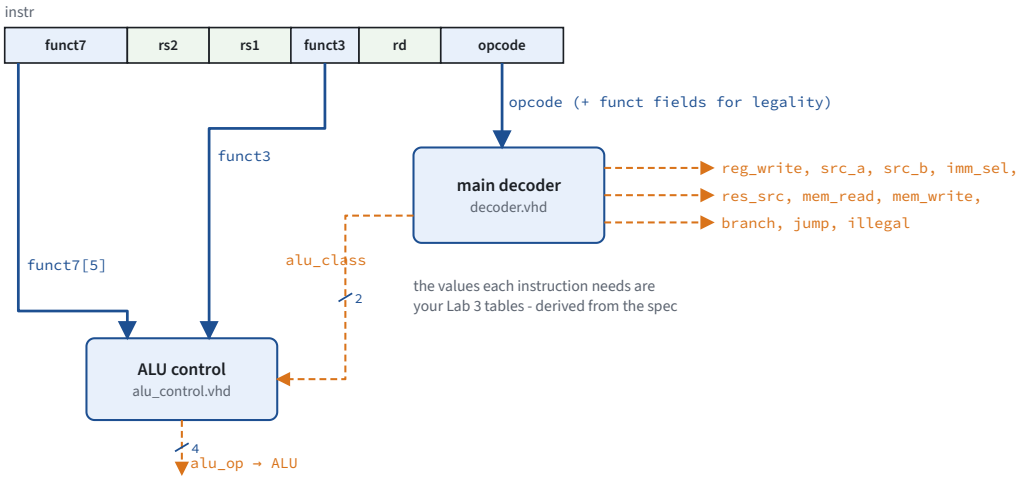


Figure 12: Two-level decoding. The values in each table are what you derive in this lab.

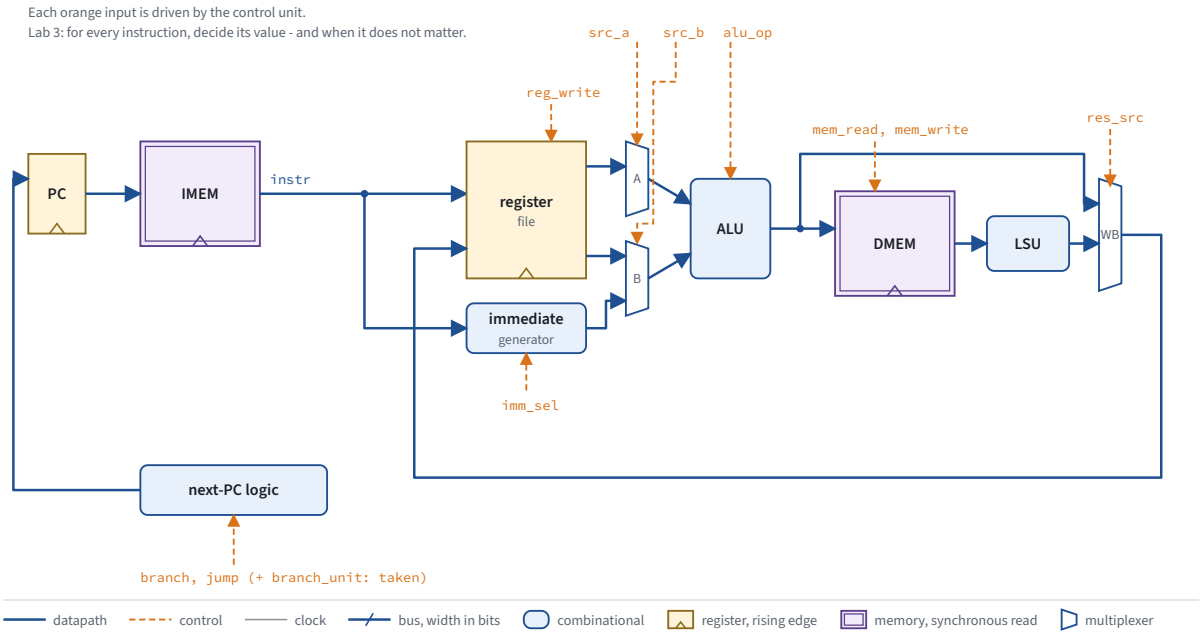


Figure 13: The datapath's control points. Each orange input is driven by the control unit.

- The ALU is the only adder in the datapath. It must also compute load and store addresses, branch and jump targets and the results of `LUI` and `AUIPC`. Which operands does each need?
- A branch compares `rs1` with `rs2` (in a separate branch unit, built in Lab 4) while the ALU computes the target.
- `JAL` and `JALR` write the address of the next instruction to `rd`.
- `FENCE` has no effect on a single-hart, in-order core, but it is a legal instruction: any `fm`, `pred`, `succ`, `rs1` and `rd` values are legal and it writes no register. `FENCE.I` (Zifencei) is not part of this core.
- Whether the decoder drops `reg_write` when `rd` is `x0` is your choice: the register file ignores the write either way, and the core (Lab 4) reports no write for `x0`.
- This core does not implement `ECALL`, `EBREAK` or CSR instructions: they must raise `illegal`, and the core will halt on them (Lab 4).

Design rule

Safe defaults. Start the decoder process by assigning every output the value of an instruction that changes no state. Then override per opcode. An instruction you forgot is then harmless, not destructive.

Design rule

Illegal means no side effects. When `illegal` is 1, the decoder must not request a register write, a memory access, a branch or a jump. The core relies on this.

Testing without the answer key

If the provided test listed the expected control signals, it would hand you the table. Instead it checks *effects*. It drives an instruction into `sim/tb_control.vhd` (your decoder and ALU control together), passes the resulting signals through `tests/lab03/datapath_model.py`, and compares the next PC, register write and memory access with values worked out from the spec. The model encodes only what each control signal *means*; it uses your Lab 1 ALU model and your Lab 2 immediate decoder. Any table that produces the right effects passes.

Spec lookup questions

Spec lookup

1. List every RV32I opcode (bits 6..0) and the instructions that share it.
2. Which instructions use `funct7`, and which values of it are legal? For `SLLI`, `SRLI` and `SRAI`, where is the shift amount? The listing fixes `imm[11:5]` for these instructions: what should your decoder do with any other value in those bits?
3. For `ADDI`, `instr[30]` can be 1. Why must your ALU control not treat that as `SUB`?
4. What exactly does `AUIPC` add to, and where does `LUI` put its result? Which ALU operand selections do these imply?
5. What value is written to `rd` by `JAL` and `JALR`? What if `rd` is `x0`?
6. What encoding does the spec give for `NOP`, and why that one?

7. What are the encodings of `ECALL` and `EBREAK`?
8. The spec reserves some encodings as HINTs. Do any of them need special handling in your decoder, or do they just work?

Tasks

Task 3.1 Derive the main-control table

On paper, make a table with one row per instruction class (`LUI`, `AUIPC`, `JAL`, `JALR`, branches, loads, stores, OP-IMM, OP, `FENCE`) and one column per decoder output. Fill every cell; write "–" for don't care and justify it.

Done when: every cell is filled, and for each row you can trace its effect through Figure 13.

Task 3.2 Derive the ALU-control table

For each `alu_class` value, list how `funct3` and `funct7[5]` select the ALU operation.

Done when: every R-type and I-type ALU instruction maps to exactly one `ALU_*` value.

Task 3.3 Implement the main decoder

Fill in `rtl/core/decoder.vhd`, starting with safe defaults.

Done when: the decoder analyses and elaborates (`make elab`).

Task 3.4 Legality

Extend the decoder so that `illegal` is 1 for every encoding that is not an implemented RV32I instruction, including bits 1..0 $\neq$ `11`, and that no state change is requested for those encodings.

Done when: you have listed in your lab book every legality check and the spec sentence or listing row that justifies it.

Task 3.5 Implement the ALU control

Fill in `rtl/core/alu_control.vhd`. Run `make test LAB=3 TESTCASE=directed_effects`.

Done when: `directed_effects` passes under GHDL and NVC.

Task 3.6 Your own tests

Implement `illegal_encodings` in `tests/lab03/test_control.py`. Then add a random test (for example `random_effects`): build random legal instructions with your `encode_imm`, compute their expected effect from the *instruction semantics* in the spec (next PC, value written to `rd`, memory address), and check them the same way `directed_effects` does. Never derive the expectation from control signals: that would test your table against itself.

Done when: `make test LAB=3` passes, and your random test has run at least 2000 instructions.

Verification checkpoint

Verification checkpoint

`make test LAB=3` passes under GHDL and NVC: `directed_effects` (provided), `illegal_encodings` and your random test.

Deliverables and marking

- Spec lookup answers (2 marks).
- Both control tables, with justified don't-cares (3 marks).
- `decoder.vhd` and `alu_control.vhd` (3 marks).
- Illegal-encoding and random tests (2 marks).

Extension

Count the logic levels in your decoder's critical path (from `instr` to `alu_op`) in your vendor's timing report. Try one-hot internal classification against a direct `case` and compare.

LAB 4

Integration: a working core

Objectives

By the end of this lab you will be able to:

1. explain why FPGA block RAM conflicts with a textbook single-cycle design, and compare four ways of resolving it by CPI, critical path and complexity;
2. implement instruction and data memories from vendor inference templates;
3. integrate the core: next-PC logic, branch unit, load/store unit, a one-cycle load sequencer, traps and an RVFI-style trace port;
4. hand-assemble a short program from the spec's encodings and run it on your core, matching a reference trace instruction by instruction.

Pre-lab reading

From *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508, chapter *"RV32I Base Integer Instruction Set, Version 2.1"*: *"Unconditional Jumps"*, *"Conditional Branches"* and *"Load and Store Instructions"* (in detail this time). From chapter *"Introduction"*: *"Exceptions, Traps, and Interrupts"*.

Vendor memory templates (you need the one for your board):

- AMD Vivado: *UG901 Vivado Design Suite User Guide: Synthesis*, chapter "HDL Coding Techniques", section "RAM HDL Coding Techniques" (single-port and ROM templates). Also available in Vivado under *Tools → Language Templates*.
- Intel Quartus Prime: *Quartus Prime Standard Edition User Guide: Design Recommendations*, "Recommended HDL Coding Styles", section "Inferring Memory Functions from HDL Code". Also available under *Edit → Insert Template*.

Theory

The block RAM problem

A textbook single-cycle CPU assumes memories with *asynchronous* reads: present an address, get the data in the same cycle. FPGA block RAM registers the address at the clock edge and returns the data one cycle later (Figure 14). Building 32 KB of asynchronous memory from look-up tables (Artix-7) is very expensive, and on the MAX 10, which has no LUT-RAM, it is out of the question.

There are four reasonable responses (Figure 15). This course uses the second, **fetch-ahead with a load stall**, because it keeps the single-cycle control you derived in Lab 3, runs most instructions in one cycle, maps both memories to block RAM, and leads directly to a pipeline.

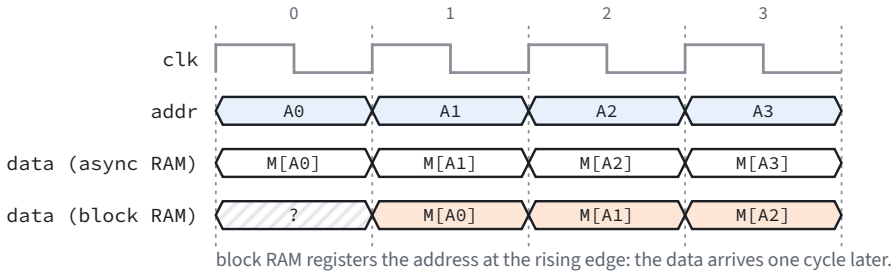


Figure 14: A textbook memory answers in the same cycle; FPGA block RAM answers in the next.

	textbook single-cycle asynchronous memories	**fetch-ahead + load stall** (this lab)	**two-phase** fetch, then execute	**multi-cycle FSM** fetch/decode/exec/mem/wb
instruction memory	async RAM: LUTs or registers	block RAM, address = pc_next	block RAM, address = pc	block RAM
data memory	async RAM	block RAM, +1 cycle on loads	block RAM	block RAM
CPI	1	1 (2 for loads)	2 (3 for loads)	3 to 5
maps to FPGA block RAM?	no	yes	yes	yes
critical path	memory + datapath	datapath + RAM address setup	datapath	shortest
route to a pipeline	conceptual only	direct: IF/EX split exists	fair	different design

Figure 15: Four ways to fit synchronous memories into a CPU.

Fetch-ahead

The trick is to address the instruction memory with the address of the *next* instruction, `pc_next`, rather than `pc` (Figure 16). At the clock edge the PC register takes `pc_next` and so does the block RAM's address register. In the following cycle the RAM's output *is* the instruction at the new PC. The block RAM's own output register plays the part of the instruction register.

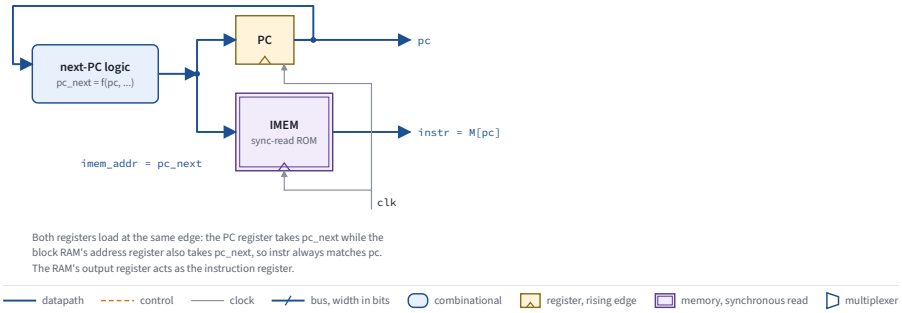


Figure 16: Fetch-ahead: both registers load `pc_next` at the same edge.

This puts the whole next-PC computation (register read, ALU, branch comparison, multiplexer) in front of the RAM's address input, so it is on the critical path. At 25 MHz that is acceptable on both boards.

Reset needs care: during reset `pc_next` must already be `RESET_VECTOR`, so that the first instruction is waiting when reset is released.

Loads take two cycles

The data memory is also synchronous: a load presents its address in one cycle and gets the data in the next. Figure 17 shows the required behaviour. In the first cycle the load computes its address and holds the PC, so `imem_addr` is `pc` again and the instruction stays in place. In the second cycle the data arrives and is written to `rd`. Stores need no extra cycle: the RAM writes at the edge.

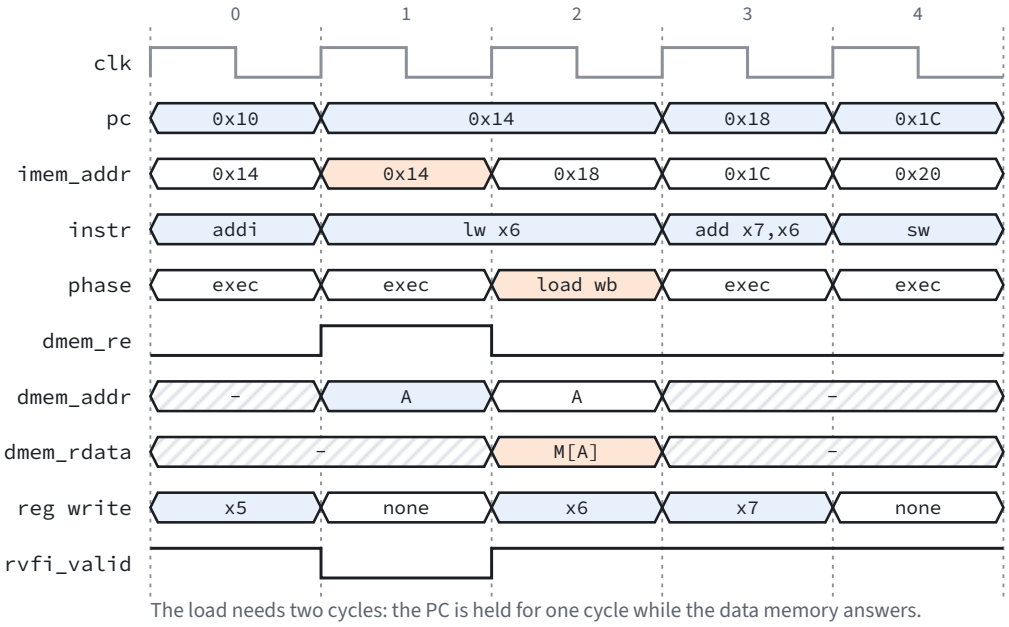


Figure 17: Timing of a load followed by a dependent instruction. Your sequencer must produce exactly this behaviour.

Next PC, branches and the load/store unit

Figure 18 lists the sources of the next PC. The ALU computes every target (`PC + imm`, or `rs1 + imm` for `JALR`); a separate branch unit decides whether a branch is taken. The load/store unit (Figure 19) moves store data onto the right byte lanes, picks the right lanes out of the loaded word and extends them.

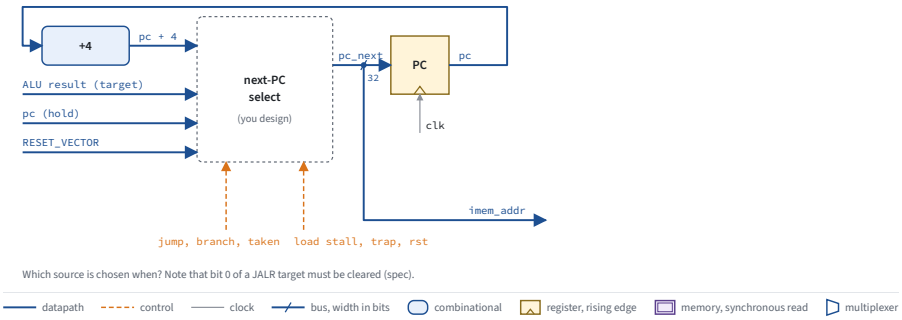


Figure 18: The sources of the next PC. Designing the selection is your task.

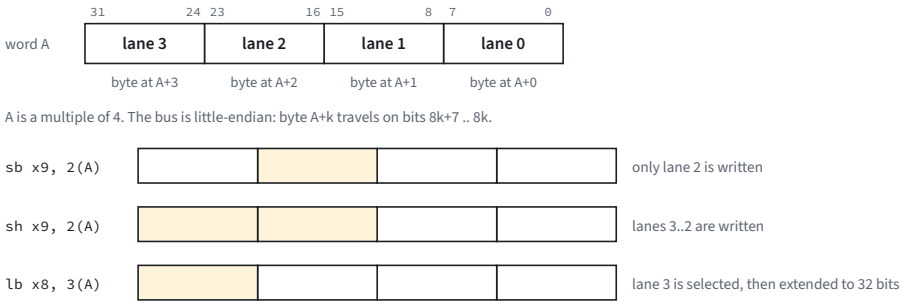


Figure 19: Byte lanes on the 32-bit little-endian data bus.

Traps

This core has no privileged architecture: no CSRs, no trap vector. When an instruction would raise an exception it instead *halts*: the trapping instruction changes no state, `trap` goes to 1 and stays there, and the trace port reports the instruction with `rvfi_trap` = 1. The core traps on illegal instructions (including `ECALL` and `EBREAK`), misaligned loads and stores, and taken branches or jumps whose target is not a multiple of four.

The trace port (RVFI)

Your core reports every instruction it completes on a set of `rvfi_*` outputs, modelled on the RISC-V Formal Interface. The tests in this lab and all of Lab 6 observe your core only through this port, so it must be exact.

Signal	Meaning in the cycle in which an instruction retires
`rvfi_valid`	1 for exactly one cycle per retired (or trapping) instruction; 0 otherwise
`rvfi_order`	0 for the first instruction after reset, then +1 per retirement
`rvfi_insn`, `rvfi_trap`	the instruction word; 1 if it trapped
`rvfi_pc_rdata`, `rvfi_pc_wdata`	its PC, and the PC of the next instruction to execute
`rvfi_rs1/2_addr`, `_rdata`	the register fields and values read (not checked)
`rvfi_rd_addr`, `rvfi_rd_wdata`	the register written and the value; both 0 if none (or if `rd` = `x0`)
`rvfi_mem_addr`	the *word* address of a load or store (bits 1..0 = 00)
`rvfi_mem_rmask`, `_wmask`	byte lanes read / written (4 bits), 0 if none
`rvfi_mem_rdata`, `_wdata`	the raw 32-bit bus data, lanes as on the bus

The `rvfi_*` outputs are *combinational*: they describe the instruction retiring in the current cycle, and the testbench samples them in the middle of the cycle (on the falling edge), once they have settled. The `trap` output is *registered*: it rises at the clock edge that ends the cycle in which the trapping instruction is reported with `rvfi_trap` = 1, and then stays at 1. The Lab 4 trace checker compares, for every instruction: `pc_rdata`, `insn`, `rd_addr`, `rd_wdata`, `mem_wmask`, `mem_addr` (for loads and stores), the written lanes of `mem_wdata`, and `trap`. Lab 6's lockstep check adds `order`, `pc_wdata` and `mem_rmask`.

Design rule

Don't read your own output ports. Assign internal signals and copy them to the ports. Reading an `out` port is legal in VHDL-2008 but not reliably supported by every tool in this

course (Appendix A).

Design rule

Use the vendor template for every memory. Infer each RAM or ROM with the exact process structure from your vendor's template. A small change, such as a reset on the RAM array or a read enable in the wrong place, can silently turn 32 KB of block RAM into 260 000 flip-flops.

Warning

The instruction memory never sees `pc`. If you connect `imem_addr` to `pc`, every instruction arrives one cycle late, and the first tests fail with a trace that is offset by one instruction.

Spec lookup questions

Spec lookup

1. How is the `JALR` target computed, and which bit is cleared? Why does the spec say this is done?
2. What is the difference between `LB` and `LBU`? Between `LH` and `LHU`? Does RV32I have an `LWU`? (Check the RV32I listing.)
3. What does the spec say about loads and stores whose address is not naturally aligned? What does this core do, and is that allowed?
4. When exactly does the spec require an instruction-address-misaligned exception for a branch? For a branch that is not taken?
5. What does a write to `x0` by a load do? Is the load still performed?
6. Which register does `JAL x0, label` write, and what is the pseudo-instruction for it?
7. By how much does the PC advance after a `BEQ` that is not taken?

Tasks

Task 4.1 Memories

Implement `rtl/mem/imem.vhd` (synchronous-read ROM) and `rtl/mem/dmem.vhd` (four 8-bit single-port RAM lanes) from your vendor's templates. Initialise them with `region_image()` from `prog_image_pkg` (read the package declaration's comments).

Done when: both analyse; you have marked in your lab book which lines come from the vendor template.

Task 4.2 Branch unit and load/store unit

Implement `rtl/core/branch_unit.vhd` and `rtl/core/lsu.vhd`. Run `make test LAB=4 K=bra`

nch_unit and make test LAB=4 K=lsu.

Done when: both unit tests pass, and you have implemented the two TODO tests in tests/lab04/test_units.py (branch_random, lsu_random) against Python models you write from the spec.

Task 4.3 Datapath

In rtl/core/rv32i_core.vhd, instantiate your blocks and build the operand and write-back multiplexers. Leave the PC and sequencer for the next task.

Done when: the core elaborates (make elab).

Task 4.4 Next PC, sequencer and traps

Draw the sequencer's state diagram and a timing diagram for a load, and check both against Figure 17. Then implement the PC register, the next-PC logic (including reset and hold) and trap handling.

Done when: your diagrams are in your lab book and the core elaborates.

Task 4.5 Trace port

Drive every rvfi_* output as specified in the table above.

Done when: make test LAB=4 K=alu passes (K selects tests by name).

Task 4.6 Run the integration programs

Run make test LAB=4. The programs and their assembly listings are in tests/lab04/programs.json. When an instruction differs, the message names it; open the waveform and find the cycle.

Done when: alu, branches, loadstore, sum_array and all six trap programs pass under GHDL and NVC.

Task 4.7 Hand assembly

Encode every instruction of sw/lab04/hand.S by hand, using only the spec's format and encoding tables, and write the words into sw/lab04/hand.hex. Show your working for at least the bne, sw, sb and jal lines.

Done when: test_hand_assembled passes.

Task 4.8 Synthesis check

Synthesise the core with the memories (soc needs Lab 7's peripherals, so make a scratch top level with the core, imem and dmem, or use ghdl --synth). Confirm that both memories became block RAM.

Done when: the report shows block RAM for IMEM and DMEM and no latches.

Verification checkpoint

Verification checkpoint

`make test LAB=4` passes under GHDL and NVC: both unit tests, the four integration programs, the six trap programs, and your hand-assembled program.

Deliverables and marking

- Spec lookup answers and the comparison of the four memory strategies in your own words (2 marks).
- State diagram and load timing diagram (2 marks).
- The core, memories, branch unit and LSU, all passing (4 marks).
- Hand assembly with working shown (1 mark).
- Synthesis evidence of block RAM inference (1 mark).

Extension

Implement the two-phase (fetch, then execute) alternative from Figure 15 as a separate architecture of `rv32i_core`. Measure the CPI of `sum_array` for both versions from the trace (cycles divided by retirements), and compare the critical paths in your timing report.

LAB 5

The software toolchain

Objectives

By the end of this lab you will be able to:

1. compile, link and disassemble C and assembly for RV32I with `riscv-none-elf-gcc`, and explain every flag you use;
2. read a linker script and an ELF section table, and explain where each section is placed in the lab SoC's Harvard memory map;
3. complete the C start-up code (`crt0.S`) and explain why each step is needed;
4. produce memory images that work unchanged in simulation and in both vendors' synthesis tools;
5. run C programs on your core and use the simulation console to debug them.

Pre-lab reading

- *RISC-V ABIs Specification* (ELF psABI), Version 1.0: *"Integer Register Convention"* and *"Integer Calling Convention"*.
- *RISC-V Assembly Programmer's Manual* (riscv-non-isa/riscv-asm-manual, commit `167ad3d`): *"Pseudo Ops"*, *"Assembler Relocation Functions"*, *"Load Immediate"*, *"Load Address"* and *"Function Calls"*.
- *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508: in chapter *"Introduction"*, the section *"Memory"* (address space, little-endian byte order).
- The memory map in Appendix B.

Theory

From C to memory images

Figure 20 shows the flow. GCC compiles each C file to an object file. The linker combines them with `crt0.o` and with `libgcc` (which supplies, for example, multiplication and division routines for a CPU without those instructions), and places every section at an address given by the linker script `sw/common/link.ld`. The result is an ELF file. `tools/gen_mem.py` then extracts the bytes for each memory.

The compiler flags matter:

`-march=rv32i_zicsr -mabi=ilp32`
: generate only RV32I instructions (Zicsr is listed because current GCC requires it to be named

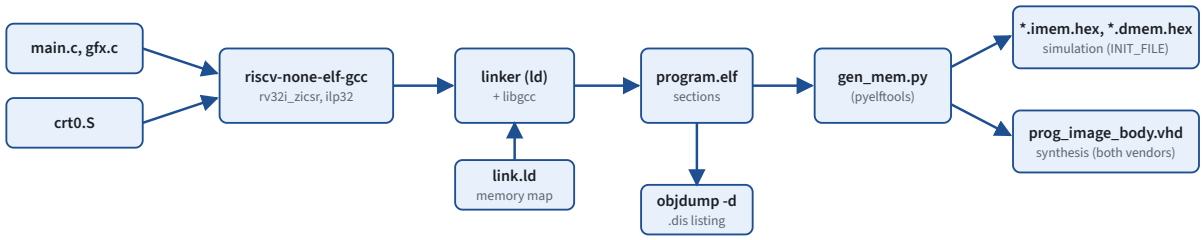


Figure 20: From C source to memory images.

before CSR instructions are accepted in assembly; our C code never uses them), with 32-bit `int`, `long` and pointers and no floating-point registers.

`-ffreestanding -nostdlib -nostartfiles`
: no operating system, no C library and no default start-up code: `crt0.S` is ours.

`-T link.ld`
: use our memory map.

Sections and the Harvard memory map

Figure 22 shows the address map. The core fetches instructions from IMEM and accesses data in DMEM; it *cannot* load from IMEM. Read-only data such as string constants and `switch` jump tables must therefore be placed in DMEM, next to `.data` (Figure 21). Both memories get their contents when the FPGA is configured, so `.data` is placed where it runs and nothing needs to copy it. `.bss` occupies no bytes in the ELF file, and the start-up code must zero it.

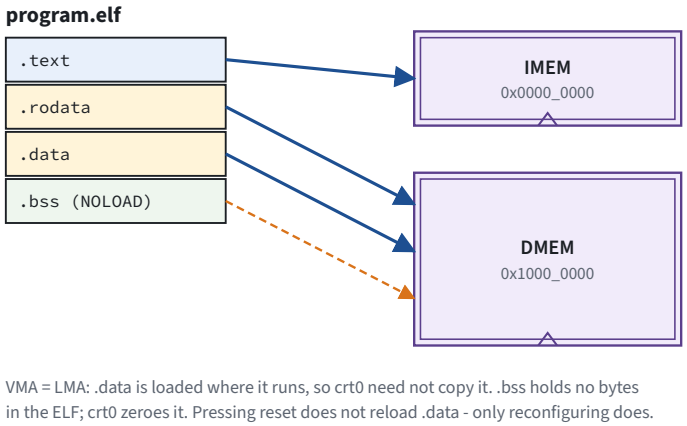


Figure 21: Where each ELF section goes.

Start-up code

Before `main` runs, `crt0.S` must set up the stack pointer and the global pointer, and zero `.bss`. When `main` returns, it writes the return value to the simulation-only `SIM_HALT` register, which ends a simulation and is ignored by the hardware. Figure 23 shows the stack frame the compiler builds on top of the stack `crt0` provides.

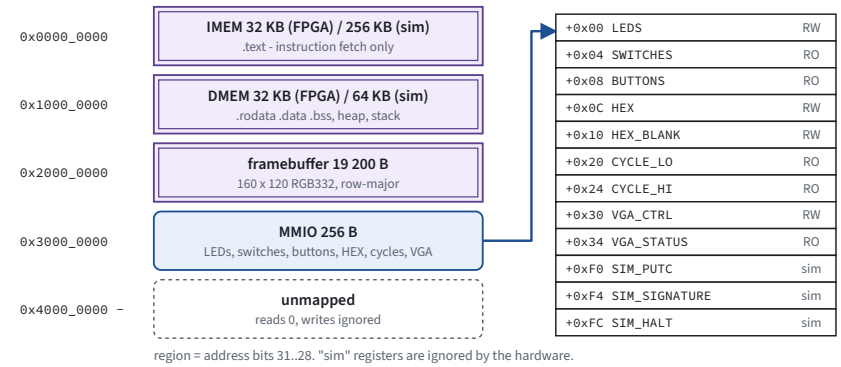


Figure 22: The SoC memory map (full details in Appendix B).

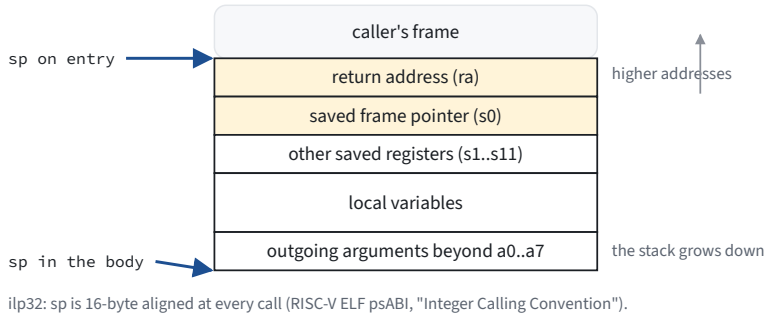


Figure 23: A function's stack frame under the ilp32 calling convention.

Memory images for simulation and synthesis

`gen_mem.py` writes three files:

- `NAME.imem.hex` and `NAME.dmem.hex`: one 32-bit word per line. The simulation-only body of `prog_image_pkg` reads these at elaboration, so the tests can switch programs without recompiling any VHDL.
- `prog_image_body.vhd`: a package body that holds the same words as VHDL constants. Both Vivado and Quartus initialise inferred RAM and ROM from a signal's or constant's initial value. One generated file therefore works for both vendors, with no `.coe`, `.mem` or `.mif` files to keep in step.

Design rule

Simulation-only files stay out of synthesis. Everything in `sim/` (including the hex-reading package body) is for simulation only. A synthesis project uses the generated `prog_image_body.vhd` instead.

Warning

Reset does not reload `.data`. Only configuring the FPGA writes the initial values. If your program changes an initialised variable and you press reset, `main` starts with the changed value. Write programs that initialise what they need at run time.

Spec lookup questions

Spec lookup

1. What alignment does the ilp32 calling convention require of `sp` at a call? Where does `crt0.S` establish it?
2. Which registers must a function preserve (callee-saved), and which may it overwrite? Where are the first eight integer arguments passed, and where is the return value?
3. What does the `la` pseudo-instruction expand to, and what do `%hi` and `%lo` mean? Why does `crt0.S` use `.option norelax` around the instruction that loads `gp`?
4. What does `li a0, 0x12345678` expand to? And `li a0, 5`?
5. In which byte order does RISC-V store a 32-bit word in memory? Which byte of the word at address 0x10000000 does `lbu` at 0x10000003 return?

Tasks

Task 5.1 Build and inspect

Run `make` in `sw/lab05`. With `riscv-none-elf-objdump -h` list the sections of `build/hello/hello.elf`; read the disassembly in `hello.dis` and the memory map in `hello.map`.

Done when: you can say, for each section, its address, its size and which memory it lives in.

Task 5.2 Check your hand assembly

Assemble `sw/lab04/hand.S` with GCC (`riscv-none-elf-gcc -march=rv32i_zicsr -mabi=ilp32 -nostdlib -Wl,-Ttext=0`), disassemble it with `-M no-aliases,numeric`, and compare it with your `hand.hex` from Task 4.7.

Done when: every word matches, or you have explained each difference.

Task 5.3 Finish the start-up code

Complete the `.bss` loop in `sw/common/crt0.S`.

Done when: it builds, and you can explain why the loop is correct when `.bss` is empty.

Task 5.4 Run C on your core

Run `make test LAB=5`. If a program does not halt, add `sim_puts` calls (`sw/common/lib.h`) to find out how far it gets; the console output is printed in the test log.

Done when: `hello` and `selftest` pass under GHDL and NVC, and `selftest` prints twelve `PASS` lines.

Task 5.5 Explain the self-test

For each `selftest` check, find the instructions that implement it in `selftest.dis`: the `.bss` loop, a `.rodata` access, the call to `__mulsi3`, the `switch` jump table, and a `lb` that sign-extends.

Done when: your lab book has one annotated excerpt per check.

Task 5.6 Your own program

Write a C program of your own (for example a sieve of Eratosthenes, a CRC32 or a small sort) that prints its result with `sim_puts` and returns 0 when the result is right. Add it to `sw/lab05`, and add it to the `EXPECT` table in `tests/lab05/test_programs.py`.

Done when: your program passes in `make test LAB=5`.

Task 5.7 Images for synthesis

Look at `build/hello/prog_image_body.vhd`. Find the constant that holds `.rodata` and the first instruction of `_start`.

Done when: you can explain how a ROM in `imem.vhd` receives these values in both simulation and synthesis.

Verification checkpoint

Verification checkpoint

`make test LAB=5` passes under GHDL and NVC for `hello`, `selftest` and your own program.

Deliverables and marking

- Spec lookup answers (2 marks).
- Section table with addresses, and the hand-assembly comparison (2 marks).
- `crt0.S` with the `.bss` loop (2 marks).
- Annotated `selftest` excerpts (2 marks).
- Your own program and its test (2 marks).

Extension

Add a heap: implement a minimal `malloc` (a bump allocator from `__heap_start` up to a limit below the stack) and a test that uses it. Explain how you chose the limit.

LAB 6

Verification in depth

Objectives

By the end of this lab you will be able to:

1. write an instruction-set simulator (ISS) in Python from the spec, and use it as a golden model;
2. check every instruction your core retires against the ISS in lockstep, through the RVFI trace port;
3. generate constrained-random programs that are always legal and always terminate;
4. define functional coverage bins, measure coverage and close it;
5. run the official RV32I architecture tests (riscv-arch-test) and explain what they do and do not prove.

Pre-lab reading

- *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508: the whole of chapter *"RV32I Base Integer Instruction Set, Version 2.1"* again, and the RV32I table in *"RV32/64G Instruction Set Listings"*. The ISS is the spec in executable form.
- The riscv-arch-test `README.md` (link in the repository README): what a "signature" is.

Theory

Golden models and lockstep

Labs 1 to 5 compared your design with fixed expectations. Lockstep comparison is stronger. An independent model, the ISS, executes the same program. After every instruction your core retires, the ISS executes one instruction and the two records are compared (Figure 24). The first difference stops the test and shows both records and the instructions before them. A bug is caught in the cycle it happens, not thousands of cycles later when a wrong value finally reaches an output.

The comparison uses the trace port (Figure 25). A record is taken at each rising edge where `rvfi_valid` is 1. Values read from MMIO (switches, the cycle counter) cannot be predicted, so in lockstep mode the ISS is *given* the value the core read.

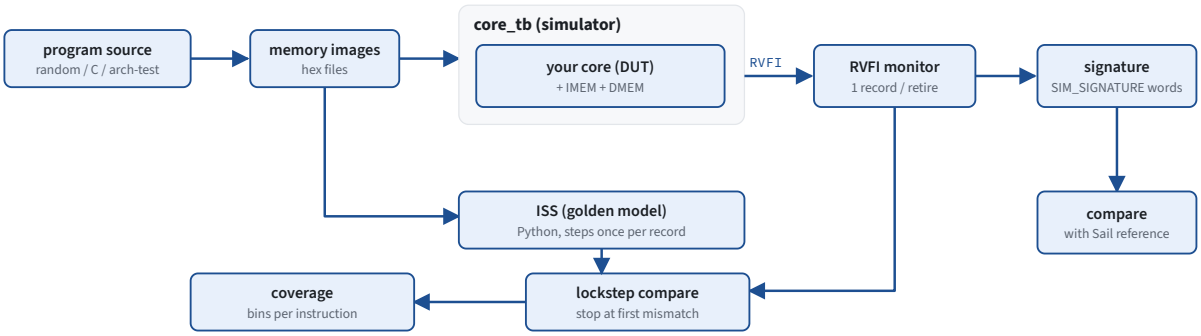


Figure 24: The verification environment for this lab.

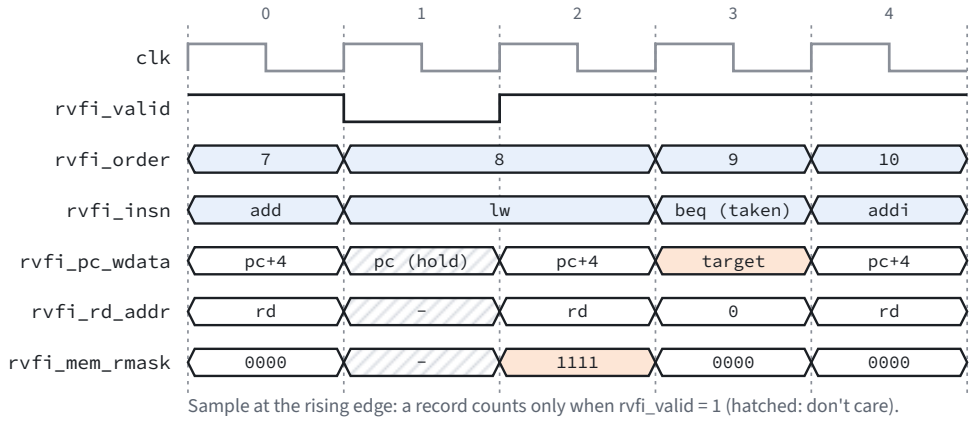


Figure 25: The RVFI port over five cycles, including a two-cycle load and a taken branch.

Constrained-random programs

Random instruction words are almost all illegal, and random branches loop forever. A constrained generator chooses *legal* instructions with random registers and immediates, but enforces rules that keep the program meaningful: loads and stores use a protected base register that points into DMEM, offsets keep accesses aligned, and branches and jumps only go forward. Within those rules, bias the choices towards interesting cases such as equal operands, `x0`, extreme values and taken branches.

Functional coverage

How do you know the random programs exercised everything? Define coverage *bins*: things that must happen at least once, such as "`SRA` executed", "`SRA` with a negative operand" or "`BLTU` taken", and count hits. Bins are the specification of your verification. The useful ones are those that would expose a real bug.

riscv-arch-test

The RISC-V architecture tests are a community suite, one test per instruction. Each test executes many cases and stores results in a *signature* region of memory. At the end, `arch-test/env/soc/model_test.h` streams the signature out through the simulation-only `SIM_SIGNATURE` register, and the test compares it with a reference signature produced by the official Sail model (Figure 26). Passing them shows your core implements the spec's behaviour on the tested cases; it does not prove the absence of bugs (for example, timing-dependent bugs in your sequencer, or trap behaviour, which the RV32I suite does not test).

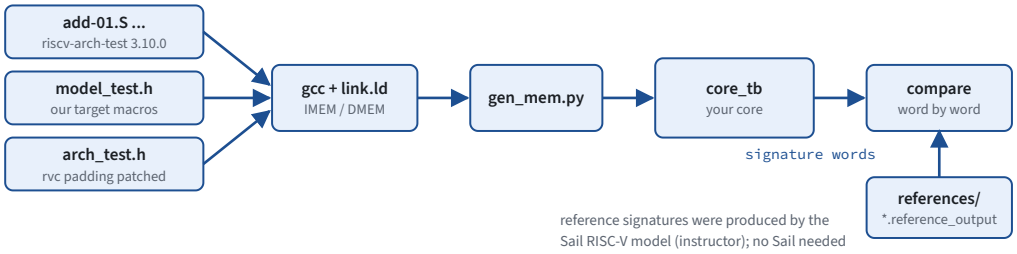


Figure 26: Building and running riscv-arch-test.

Warning

A model must not share your misunderstanding. Write the ISS from the spec, not from your VHDL, and preferably without looking at your VHDL. When the ISS and the core disagree, re-read the spec before deciding which is wrong.

Spec lookup questions

Spec lookup

1. List every condition under which your ISS must trap, with the spec sentence for each.
2. Which instructions are *HINT*s in RV32I, and what must the ISS do with them?

3. The spec permits `SLLI` with `imm[5]` = 1 on RV64 only. What must your ISS do with it on RV32I?
4. What exactly is the next PC of a `JALR` whose computed target is odd but becomes aligned after the low bit is cleared?
5. For each instruction, what goes into the RVFI `mem_wdata` lanes of a byte store, given how the bus replicates the byte? Why does the lockstep check compare only the enabled lanes?

Tasks

Task 6.1 Run the architecture tests

Fetch the suite (`make arch-test-fetch`) and run `make test LAB=6 K=arch`. Without lockstep, each test compares only its final signature.

Done when: all 39 tests pass, or for each failure you have found the first wrong signature word and the instruction that produced it.

Task 6.2 ISS: decode

Implement `mnemonic()` in `tests/common/labtest/iss.py`, from the RV32I listing.

Done when: for every word in `tests/lab03/control_cases.json`, your `mnemonic()` returns the instruction named in its `asm` field.

Task 6.3 ISS: execute

Implement `Hart.step()`. Validate the ISS *independently of your core* by running the arch-test programs on it alone and comparing with the reference signatures (`Hart.run()` returns the signature).

Done when: the ISS reproduces all 39 reference signatures.

Task 6.4 Lockstep

Run the earlier labs' programs in lockstep: `LOCKSTEP=1 make test LAB=5` and `LOCKSTEP=1 make test LAB=6 K=arch`.

Done when: every program runs in lockstep with no mismatch.

Task 6.5 Constrained-random generator

Implement `RandomProgram.generate()` in `randgen.py` under the constraints in its docstring. Run `make test LAB=6 K=random`.

Done when: at least 20 seeds of 600 instructions run to `SIM_HALT` in lockstep.

Task 6.6 Coverage

Define bins in `coverage.py`: at least one per instruction, plus bins for the cases you consider most

likely to hide bugs. Justify each bin category in your lab book. Run the coverage-closure test.

Done when: `test_coverage_closure` passes, or each missed bin is explained and the generator has been biased to hit it.

Task 6.7 Bug hunt

Ask a classmate (or your demonstrator) to inject a single subtle bug into a copy of your core, for example `SRA` wrong for shift amount 0, or `rd` written for a store when `rd` field $\neq 0$. Find it using only your environment.

Done when: you have found the bug and recorded which mechanism caught it first (arch test, lockstep, or coverage).

Verification checkpoint

Verification checkpoint

All 39 RV32I architecture tests pass with `LOCKSTEP=1`; 20 random seeds pass in lockstep; your coverage closes.

Deliverables and marking

- Spec lookup answers (1 mark).
- The ISS, validated against the reference signatures (3 marks).
- Random generator with documented constraints (2 marks).
- Coverage model with justified bins and a closure report (2 marks).
- Bug hunt report (2 marks).

Extension

Mutation testing: write a script that makes 20 single-line changes to a copy of your core (swap a comparison, invert a select, change a constant), runs your full Lab 6 suite on each, and reports which mutants survive. Improve your tests until none do.

LAB 7

FPGA deployment

Objectives

By the end of this lab you will be able to:

1. build a board-independent SoC with an address decoder and memory-mapped I/O, and explain why every read has the same latency;
2. generate the system clock with a PLL or MMCM behind a vendor wrapper entity, and simulate the board top level with a behavioural clock model;
3. design a reset synchroniser and input synchronisers, and explain metastability;
4. write timing constraints, read a timing report and state the worst slack of your design;
5. run a test program on the board, and show that the board top level passes the same tests in simulation.

Pre-lab reading

- The memory map, Appendix B.
- Your board's user manual: the switches, LEDs, push buttons and seven-segment displays (Terasic DE10-Lite User Manual; Digilent Basys 3 Reference Manual).
- Your vendor's timing-constraint introduction: AMD *UG903 Using Constraints* (`create_clock`, generated clocks) or Intel's *Timing Analyzer Cookbook* (`create_clock`, `derive_pll_clocks`).
- *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508: in chapter *"Introduction"*, the section *"Memory"* (what the spec calls I/O and why the execution environment defines it).

Theory

The SoC

Figure 27 shows the system. The core's data bus goes to an address decoder, which uses address bits 31..28 to pick DMEM, the framebuffer (Lab 8) or the MMIO registers. Every target returns read data *one cycle* after the access, as block RAM does, so the core's two-cycle load works the same for all of them. The decoder must therefore remember which target was accessed and use that in the next cycle to select the read data (Figure 28).

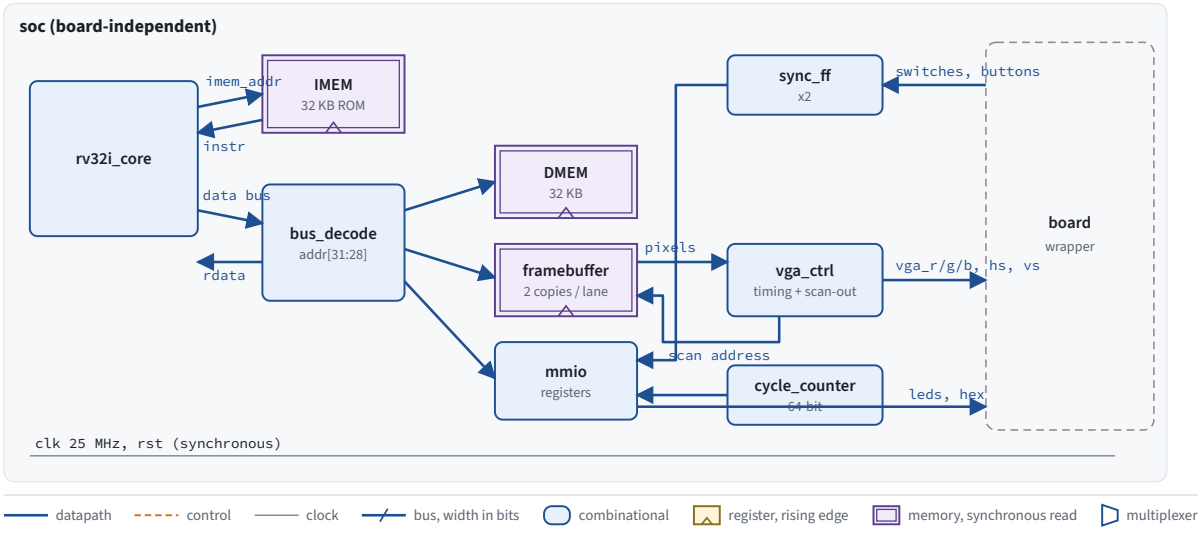


Figure 27: The board-independent SoC (`rtl/soc/soc.vhd`, provided).

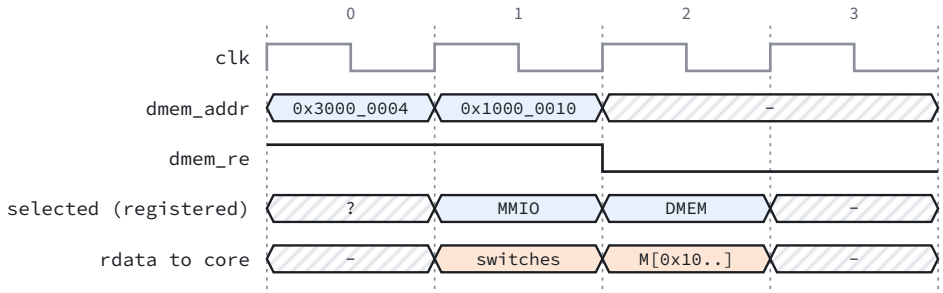


Figure 28: Registered reads: the read multiplexer selects using the region accessed in the previous cycle.

Clocking, reset and synchronisers

Both boards run the SoC at 25 MHz, which is also the VGA pixel clock in Lab 8, so the whole design has one clock domain. The clock comes from a PLL (MAX 10) or an MMCM (Artix-7). These are vendor primitives, so they live behind the `clkgen` entity, which has three architectures: one per vendor and a behavioural model for simulation (Figure 29). The board top level is otherwise identical for both vendors.

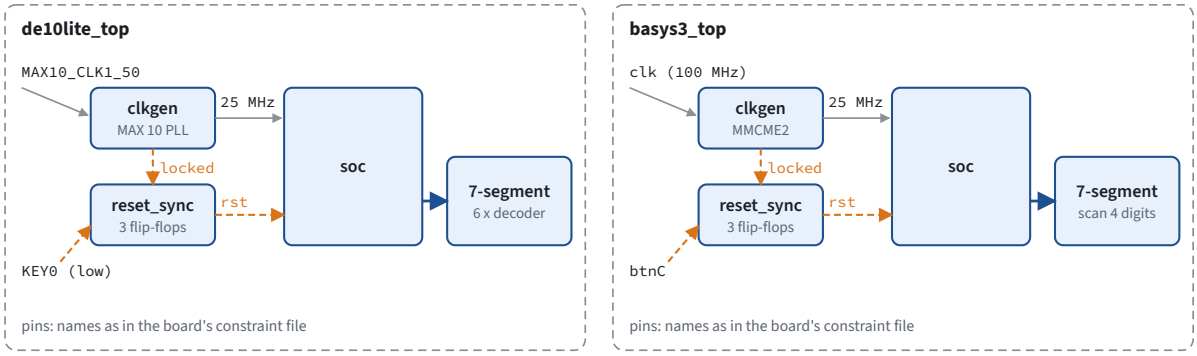


Figure 29: Both board wrappers contain the same `soc`.

Push buttons and switches are asynchronous to the clock. A flip-flop that samples a signal changing near the clock edge can become *metastable*: its output hovers between 0 and 1 for an unpredictable time. Two flip-flops in series (`sync_ff`) give the first one a full cycle to settle. The reset needs a different treatment (Figure 30). It must take effect immediately even with no clock, which means asserting asynchronously. Its *release* must be synchronous, so that every flip-flop leaves reset on the same edge.

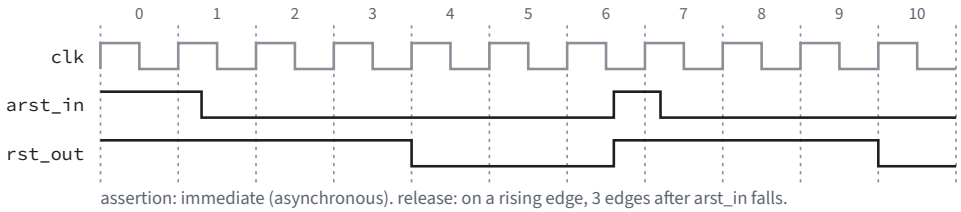


Figure 30: Reset synchroniser: asynchronous assertion, synchronous release after three edges.

Seven-segment displays

The DE10-Lite drives each of its six digits directly (active low). The Basys 3 shares the segment lines between four digits and scans them: one digit is enabled at a time, fast enough that the eye sees all four (Figure 31).

Timing

The constraint files give the board clock's period. The tools derive the 25 MHz clock from the PLL/MMCM settings and check every register-to-register path against 40 ns. The timing report's *worst negative slack* (WNS, or the setup slack in Quartus) must be ≥ 0. The longest path in this design runs from the PC register, through the register file, the ALU and the next-PC multiplexer,

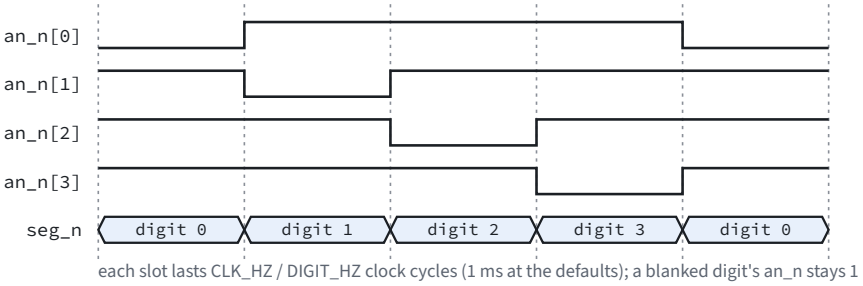


Figure 31: Scanning the Basys 3's four digits.

to the instruction memory's address register. Switches, buttons, LEDs and displays are declared false paths, because they pass through synchronisers or are only seen by people.

Design rule

One clock, one reset. Every flip-flop in the SoC uses `clk` and the synchronous `rst` from the reset synchroniser. Only the reset synchroniser itself has an asynchronous input.

Design rule

Every asynchronous input passes through a synchroniser before any logic sees it.

Design rule

Vendor primitives only behind wrapper entities. `clkgen` is the only place a PLL or MMCM appears. The rest of the design is vendor-neutral VHDL.

Spec lookup questions

Spec lookup

1. The spec says nothing about where I/O devices are. Why? Which document defines the memory map for our system?
2. For the cycle counter, the spec's *"'Zicntr' and 'Zihpm' Extensions for Counters"* describes the `cycle`/`cycleh` CSRs and a loop for reading a 64-bit counter on RV32. Find it, and explain why `read_cycles()` in `mmio.h` uses the same hi–lo–hi pattern for our MMIO counter.
3. Our core uses a store to an MMIO register to light an LED. Must the `FENCE` instruction be used between two such stores? (Consider a single hart and our in-order bus.)

Tasks

Task 7.1 Synchronisers

Implement `rtl/periph/sync_ff.vhd` and `rtl/periph/reset_sync.vhd`. Run `make test LAB=`

`7 K=reset_sync`.

Done when: `test_reset_sync` passes.

Task 7.2 Seven-segment decoder

Derive the table for `rtl/periph/sevenseg_dec.vhd` from your board's display diagram and implement it. Extend `sevenseg_patterns` in `tests/lab07/test_periph.py` with the thirteen digits it does not check.

Done when: `make test LAB=7 K=sevenseg_dec` passes with all sixteen digits checked.

Task 7.3 MMIO and the cycle counter

Implement `rtl/periph/cycle_counter.vhd` and `rtl/periph/mmio.vhd` from the memory map.

Done when: both analyse and elaborate (`make elab`).

Task 7.4 Address decoder

Implement `rtl/soc/bus_decode.vhd`. Then run `make test LAB=7 K=mmio`, which runs `sw/lab07/board_test.c` on `soc_tb`.

Done when: `test_mmio` passes.

Task 7.5 Display scanning (Basys 3)

Implement `rtl/periph/sevenseg_mux.vhd`. DE10-Lite users implement it too: it is tested in simulation.

Done when: `make test LAB=7 K=sevenseg_mux` passes.

Task 7.6 Board wrapper

Implement the wrapper for your board in `boards/<board>/<board>_top.vhd`. Both wrappers are tested in simulation with the behavioural clock model: `make test LAB=7 K=top`.

Done when: the test for your board's wrapper passes. (Do the other board's too if you have time: same SoC, different wrapper.)

Task 7.7 Constraints and timing

Read your board's constraint files (`boards/<board>/`) and check three pin assignments against the board manual. Build the bitstream (`make quartus` or `make vivado`, then compile in the GUI or with the script's `compile`/`build` option). Open the timing report.

Done when: you have recorded the worst slack and the start and end points of the critical path, and checked that the path matches the description in the theory section.

Task 7.8 On the board

Program the board with `board_test`. Check: switches appear on the LEDs and the display; button 0 (DE10-Lite KEY1, Basys 3 btnU) increments the count (DE10-Lite digits 5..4); the reset button restarts the program.

Done when: your demonstrator has seen it working.

Verification checkpoint

Verification checkpoint

`make test LAB=7` passes under GHDL and NVC (reset synchroniser, decoder, display scanning, MMIO through the SoC, and both board wrappers), and the design runs on your board with non-negative slack.

Deliverables and marking

- Spec lookup answers and a paragraph on metastability (2 marks).
- Peripherals, decoder and wrapper, passing (3 marks).
- Constraint check and timing report: worst slack and critical path (2 marks).
- Board demonstration (3 marks).

Extension

Debounce button 0 in software: sample it every millisecond (using the cycle counter) and accept a change only after it has been stable for 10 ms. Measure on the board how many false counts the undebounced version produces.

LAB 8

Graphics on the CPU

Objectives

By the end of this lab you will be able to:

1. generate 640 × 480 at 60 Hz VGA timing and scan out a framebuffer through a pipeline whose stages stay aligned;
2. build a framebuffer with two read ports from simple dual-port RAMs only, and justify the memory cost;
3. draw on the screen from software, either driven by switches (path a) or with your own graphics primitives in C (path b);
4. measure the cycle cost of software routines with a hardware cycle counter, and use the results to argue for the M extension and for caches.

Pre-lab reading

- The VGA timing summary in Figure 32, and your board manual's VGA section (resistor-ladder DAC, 4 bits per colour).
- *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508: *"M Extension for Integer Multiplication and Division, Version 2.0"*, section *"Multiplication Operations"*. You will not implement it; you will quantify what it would save.
- Path (b) only: the description of Bresenham's line algorithm in Figure 37.

Theory

VGA timing

A VGA monitor draws 525 lines of 800 pixel clocks each, of which 640 × 480 are visible (Figure 32). Between lines and between frames, a front porch, an active-low sync pulse and a back porch give the monitor time to return its beam (or, today, to lock its timing). At 25 MHz a frame takes 16.8 ms: 59.5 Hz, close enough to 60 Hz for every monitor tested so far.

The framebuffer

A 640 × 480 image at 12 bits per pixel would need 460 KB, more block RAM than either FPGA has. Instead the framebuffer holds 160 × 120 pixels of one byte each, in RGB332 format (Figure 33), which is 19 200 bytes. Each pixel is repeated 4 × 4 on screen. RGB332 is expanded to 4 bits per channel by repeating its top bits, so 0 stays 0 and full scale stays full scale.

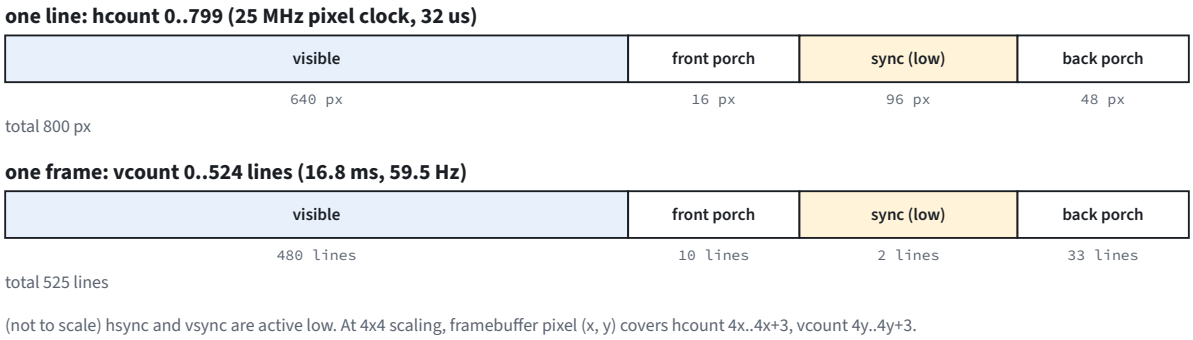


Figure 32: 640 × 480 at 60 Hz timing. Counters start at 0 at the first visible pixel.

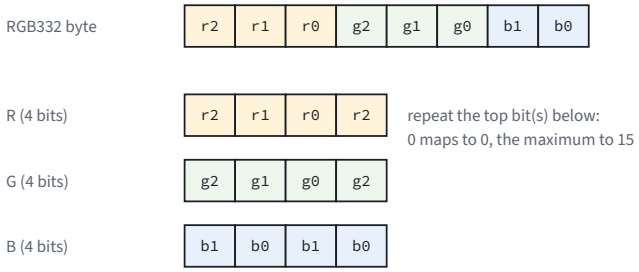


Figure 33: Expanding RGB332 to RGB444 by bit replication.

The CPU writes pixels, the VGA scan-out reads them, and blending needs the CPU to read them too. That is two read ports and one write port. The portable answer is to store every byte lane *twice*, in two simple dual-port RAMs that share the write port (Figure 34). It doubles the framebuffer's block RAM, which both boards can afford (Appendix B), and it avoids true dual-port RAM inference, whose templates are less portable.

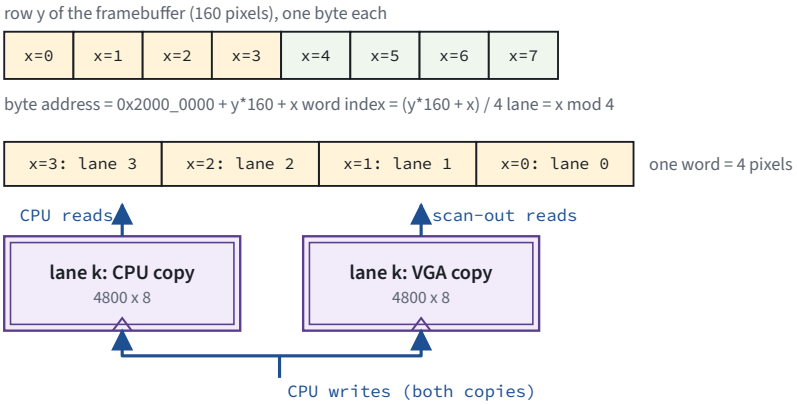


Figure 34: Framebuffer addressing and the two copies of each byte lane.

The scan-out pipeline

The RAM answers one cycle after the address. So the colour reaches the pins one cycle after the counters that produced the address, plus however many register stages you add. The sync pulses and blanking come straight from the counters and must be delayed by exactly the same number

of cycles, or the image shifts sideways (Figure 35).

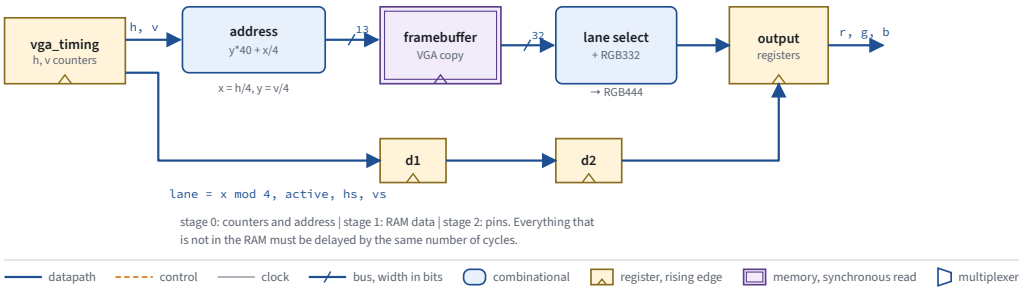


Figure 35: The scan-out pipeline. Everything not stored in the RAM is delayed to stay aligned with the pixel data.

Software graphics without a multiplier

RV32I has no multiply instruction. Pixel addresses use `y * 160`, which is `(y << 7) + (y << 5)`: two shifts and an add. Rotation needs fractional values: Q16.16 fixed point stores a value times 65 536 in an `int32_t` (Figure 36). A Q16.16 multiply takes the 64-bit product and keeps bits 47..16. Without a `MUL` instruction you build the product by shift-and-add, one bit at a time.

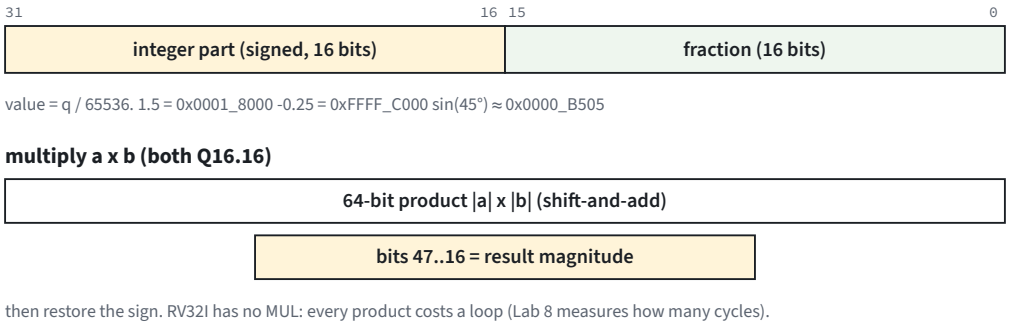


Figure 36: Q16.16 fixed point and its multiplication.

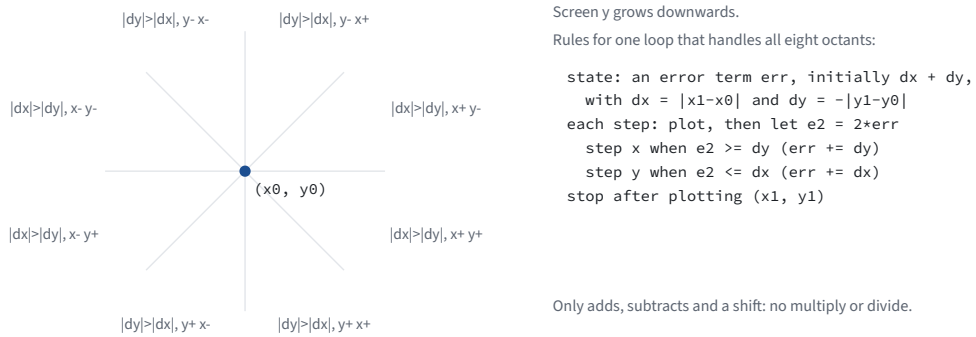


Figure 37: Bresenham's line algorithm: one integer loop for all eight octants.

Measuring

The MMIO cycle counter (`CYCLE_LO`/`CYCLE_HI`) counts clock cycles since reset. Read it before and after a routine, and the difference is the routine's cost in cycles, including load stalls. `sw/lab08/bench.c` does this for every primitive.

Choose your path

Both paths build the same VGA hardware (Tasks 8.1 to 8.4).

Path (a): switch-driven graphics
`sw/lab08/switch_demo.c` draws a shape chosen by the switches, in a colour set by the switches, at a size changed with button 0 (the file header gives the exact behaviour, which the golden images check). The display shows the parameters.

Path (b): graphics primitives in C
implement `gfx.c`, `fixed.c` and `cube.c`: fast clears and rectangles with word stores, Bresenham lines, Q16.16 multiply by shift-and-add, alpha blending, and a rotating wireframe cube. Then measure every primitive.

Path (b) is the more demanding and is marked accordingly (see Deliverables).

Design rule

Match the reference algorithms exactly (path b). The golden images were produced with the rules in `gfx.c`, `fixed.c` and `cube.c`: Bresenham with `err = dx + dy`; per-channel blend `(dst*(8-alpha) + src*alpha) >> 3`; Q16.16 products truncated towards zero. A different but valid algorithm can be off by one pixel, and a pixel-exact test will report that.

Spec lookup questions

Spec lookup

1. Which instructions does the M extension add for multiplication? Which one returns the low 32 bits, and which the high 32 bits of a signed × signed product?
2. The spec recommends an instruction sequence for when both the high and the low half of a product are needed. What is it, and why does the order matter to a microarchitecture?
3. How would `q16_mul` be written with M-extension instructions? How many instructions would it take?

Tasks

Task 8.1 VGA timing

Implement `rtl/periph/vga_timing.vhd`. Run `make test LAB=8 K=vga_timing`.

Done when: `test_vga_timing` passes: every line and the whole frame have the right structure.

Task 8.2 Simple dual-port RAM

Implement `rtl/mem/ram_sdp.vhd` from your vendor's simple dual-port template.

Done when: it analyses, and you have checked your template's read-during-write behaviour against the header comment and explained why the framebuffer never relies on it.

Task 8.3 Framebuffer

Implement `rtl/mem/framebuffer.vhd` with eight `ram_sdp` instances.

Done when: it elaborates in `soc_tb` (`make elab`).

Task 8.4 Scan-out

Implement `rtl/periph/vga_ctrl.vhd`. Draw its pipeline with the cycle in which each signal is valid.

Done when: your pipeline drawing shows sync, blanking and colour aligned at the pins.

Task 8.5 Fill the screen

Path (a): implement the clearing and the filled-square shape in `switch_demo.c`. Path (b): implement `fb_clear` and `put_pixel`. Run `make test LAB=8` and look at the captured images (`*.capture.png` in `sim_build/<sim>/soc_tb/`).

Done when: the captured image shows your colours in the right places, even if the tests still fail on details.

Task 8.6 Rectangles and lines

Path (a): finish all four shapes and the button and display behaviour. Path (b): implement `fill_rect` (word stores in the middle of each row) and `draw_line`.

Done when: path (a): `switch_demo_outline` and `switch_demo_filled` pass. Path (b): most of `gfx\_test`'s capture matches the golden image (only the blended area and the cube differ).

Task 8.7 Fixed point (path b)

Implement `umul32x32`, `q16_mul` and `mul_small` in `fixed.c` with shifts and adds only. Test them on the host first: compile `fixed.c` with your native C compiler and compare with a few hand-computed products.

Done when: your host tests pass for positive, negative and zero operands.

Task 8.8 Blending (path b)

Implement `blend332` and `blend_rect`.

Done when: the blended area of `gfx\_test` matches.

Task 8.9 The cube (path b)

Implement `draw_cube` and `wait_frame`.

Done when: `gfx_test` and `cube_demo` pass.

Task 8.10 Measure

Run `make test LAB=8 K=bench` and read the cycle counts in the log. For path (a), also time one full redraw of your demo with the cycle counter.

Done when: you have a table of cycle counts, and for each routine, cycles per pixel (or per operation).

Task 8.11 On the board

Build the bitstream with your demo (`cube_demo` or `switch_demo`): pass the demo's `prog_image_body.vhd` to the project script.

Done when: your demonstrator has seen it on a monitor.

Verification checkpoint

Verification checkpoint

`test_vga_timing` passes; path (a): both `switch_demo` golden-image tests pass; path (b): `gfx_test` and `cube_demo` golden-image tests and `test_bench` pass. All under GHDL and NVC.

Deliverables and marking

- VGA hardware: timing, framebuffer, scan-out, with your pipeline drawing (3 marks).
- Path (a), up to 4 marks: the demo, passing its golden images, plus a measurement of one redraw. Path (b), up to 5 marks: primitives, cube and benchmark, passing all golden images.
- Discussion, about one page (2 marks). Using your measurements: how many cycles would an M-extension multiplier save in `blend_rect` and in `draw_cube`? Would a data cache speed up `fb_clear`? Why or why not?
- Board demonstration (path a: 1 mark; path b: included above).

Extension

Double buffering: the framebuffer region has room for one frame only. Propose (and, if you like, implement) a change that removes tearing in `cube_demo`, and estimate its block-RAM cost on your board.

APPENDIX A

The portable VHDL subset

The same RTL must analyse in GHDL and NVC and synthesise in AMD Vivado and in Intel Quartus Prime Lite. Every source file is compiled as VHDL-2008, but Quartus Prime Standard and Lite support only part of VHDL-2008. Intel's help page *Quartus Prime Support for VHDL 2008* lists the supported additions: unconstrained array elements, the matching operators (`?=` and so on), the condition operator `??`, matching `case?`, simplified sensitivity lists (`process(all)`), `elsif`/`else` and `case` in generate statements, enhanced bit-string literals and block comments. Vivado supports a larger set (AMD UG901, "VHDL-2008 Language Support").

The rule for this course: write VHDL-93, plus `process(all)`. Everything in the table below is used in the reference design and is VHDL-93 unless marked otherwise.

Use freely	Notes
`ieee.std_logic_1164`, `ieee.numeric_std`	the only arithmetic packages
`process(all)`	VHDL-2008, listed by Intel
`rising_edge(clk)` in clocked processes	no `clk'event` forms
concurrent `when`/`else` and `with`/`select`	outside processes
`case`, `if` and `for` loops with constant bounds in processes	
records and arrays of `std_logic_vector` for *internal* signals	keep ports simple (below)
`for`/`if` generate, with local signal declarations	
generics of type `natural`, `positive`, `string`, `boolean`	
functions (including `impure` functions for memory initialisation)	

Avoid in synthesisable files	Why
reading an `out` port inside the architecture	VHDL-2008 only; not in Intel's list. Use an internal signal.
conditional (`when`/`else`) or selected assignment *inside* a process	VHDL-2008 only; not in Intel's list
expressions as port-map actuals (`port map (a => x and y)`)	VHDL-2008 only; use a signal
`?=`, `??`, `case?`	listed by Intel, but easy to misuse; never needed here
generic types, generic packages, external names, `force`	not supported by Quartus Lite
`std_logic_arith`, `std_logic_unsigned`	non-standard
`to_hstring`, `'image`, file I/O	simulation only; wrap in `-- synthesis translate_off/on` or keep in `sim/`
shared variables	not allowed without a protected type in VHDL-2008 (GHDL and NVC reject them)
records or enumerated types in *ports* of tested entities	cocotb on GHDL cannot access them reliably
asynchronous reset of a RAM array	prevents block-RAM inference

`tools/vhdl_lint.py` in your repository checks the common cases: `python3 tools/vhdl_lint.py rtl boards`.

APPENDIX B

Memory map and I/O registers

Region selection uses address bits 31..28. Accesses to unmapped addresses read zero and write nothing (in simulation, a warning is printed). Every data read returns its value one cycle after the access.

Base	Region	Contents
`0x0000_0000`	IMEM	32 KB on the FPGA (256 KB in simulation); `.text`. Instruction fetch only: data accesses read 0.
`0x1000_0000`	DMEM	32 KB on the FPGA (64 KB in simulation); `.rodata`, `.data`, `.bss`, heap, stack. Aliased modulo its size.
`0x2000_0000`	framebuffer	19 200 bytes: 160 × 120 RGB332 pixels, row-major. Bytes beyond read 0.
`0x3000_0000`	MMIO	word registers, below

Offset	Name	Access	Description
`0x00`	`LEDS`	RW	bits 15..0 drive the LEDs (DE10-Lite shows 9..0)
`0x04`	`SWITCHES`	RO	bits 15..0, synchronised (DE10-Lite has 9..0)
`0x08`	`BUTTONS`	RO	bits 3..0, 1 = pressed. Bit 0: DE10-Lite KEY1, Basys 3 btnU; bits 1–3: Basys 3 btnL, btnR, btnD
`0x0C`	`HEX`	RW	eight hexadecimal digits, digit 0 = bits 3..0 (DE10-Lite shows 6, Basys 3 shows 4)
`0x10`	`HEX_BLANK`	RW	bits 7..0: 1 turns the digit off
`0x20`	`CYCLE_LO`	RO	cycle counter bits 31..0
`0x24`	`CYCLE_HI`	RO	cycle counter bits 63..32 (read hi, lo, hi)
`0x30`	`VGA_CTRL`	RW	bit 0: 1 enables video output
`0x34`	`VGA_STATUS`	RO	bit 0: in vertical blanking; bits 31..16: frame counter
`0xF0`	`SIM_PUTC`	WO	simulation console: bits 7..0 are printed
`0xF4`	`SIM_SIGNATURE`	WO	simulation: one signature word (riscv-arch-test)
`0xFC`	`SIM_HALT`	WO	simulation: ends the run; the value is the exit code

The three simulation-only registers are ignored by the hardware. The testbenches observe them through the RVFI trace port. The reset button is KEY0 on the DE10-Lite and btnC on the Basys 3; it is not visible to software.

Block-RAM budget (estimates; vendor tools may pack differently): IMEM and DMEM 32 KB each, framebuffer 2 × 19 200 bytes. That is about 104 of 182 M9K blocks on the MAX 10 10M50, and about 28 of 50 RAMB36 equivalents on the Artix-7 XC7A35T, leaving headroom for larger programs.

APPENDIX C

Reference documents and versions

Section numbers change between releases of the RISC-V specifications, so this manual cites sections by *title*. Every title cited is checked automatically against these exact versions (instructor tooling, not part of your repository).

***The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508**
RISC-V International, ratified release tagged `20250508` in `https://github.com/riscv/riscv-isa-manual`, file `riscv-unprivileged-20250508.pdf`. Main chapters used: *"Introduction"*; *"RV32I Base Integer Instruction Set, Version 2.1"*; *""Zicntr" and "Zihpm" Extensions for Counters"*; *"M Extension for Integer Multiplication and Division, Version 2.0"*; *"RV32/64G Instruction Set Listings"*.

***RISC-V ABIs Specification* (ELF psABI), Version 1.0**
RISC-V International, tag `v1.0` of `https://github.com/riscv-non-isa/riscv-elf-psabi-doc`. Sections used: *"Integer Register Convention"*, *"Integer Calling Convention"*.

***RISC-V Assembly Programmer's Manual* (riscv-non-isa/riscv-asm-manual, commit `167ad3d`)**
`https://github.com/riscv-non-isa/riscv-asm-manual`, commit `167ad3d022afd7480b629ccda08c9f61cc8cfa5a` (the manual is not versioned). Sections used: *"Pseudo Ops"*, *"Assembler Relocation Functions"*, *"Load Immediate"*, *"Load Address"*, *"Function Calls"*.

riscv-arch-test
tag `3.10.0` of `https://github.com/riscv/riscv-arch-test`, suite `riscv-test-suite/rv32i\_m/I` (39 tests). Reference signatures were generated with the Sail RISC-V model, release 0.16.

Board documentation
Terasic *DE10-Lite User Manual*; Digilent *Basys 3 Reference Manual* and `Basys-3-Master.xdc` (github.com/Digilent/digilent-xdc).

Vendor documentation
AMD UG901 (synthesis, RAM templates, VHDL-2008 support), AMD UG903 (constraints); Intel *Quartus Prime Standard Edition User Guide: Design Recommendations* (memory inference) and *Quartus Prime Support for VHDL 2008*.